

UNIT - V

File System Structure – File System Implementation – Directory Implementation – Allocation Methods – Free-space Management. Kernel I/O Subsystems - Disk Structure – Disk Scheduling – Disk Management – Swap-Space Management.

Case Study: The Linux System, Windows.

2 MARKS

1. Define Bootstrap Program?(NOV'12)

Bootstrapping usually refers to the starting of a self-sustaining process that is supposed to proceed without external input. In computer technology the term (usually shortened to booting) usually refers to the process of loading the basic software into the memory of a computer after power-on or general reset, especially the operating system which will then take care of loading other software as needed.

2. What is boot control block?

A boot control block (per volume) can contain information needed by the system to boot an operating system from that volume. If the disk does not contain an operating system, this block can be empty. It is typically the first block of a volume. In UFS, it is called the boot block: in NTFS, it is the boot block in NTFS, it is the partition block sector .

3. What is volume control block?

A volume control block (per volume) contains volume (or partition) details, such as the number of blocks in the partition, the size of the blocks, a free-block count and free-block pointers, and a free-FCB count and FCB pointers. In UFS, this is called a superblock in NTFS, it is stored in the master file table

4. Draw file control block

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks

5. List the methods to implement directory

Linked list

The simplest method of implementing a directory is to use a linear list of file names with pointers to the data blocks. This method is simple to program but time-consuming to execute.

Hash table

In this method, a linear list stores the directory entries, but a hash data structure is also used. The hash table takes a value computed from the file name and returns a pointer to the file name in the linear list. Therefore, it can greatly decrease the directory search time.

6. Write the types of directory allocation methods?

Three major methods of allocating disk space are

1. Contiguous allocation.
2. linked allocation.
3. indexed allocation.

7. What is Contiguous Allocation?

A single contiguous set of blocks is allocated to a file at the time of file creation; this is pre allocation strategy that uses portions of variable size. The FAT (File Allocation Table) needs just a single entry for each file, showing the starting block and the length of the file. Disk addresses define a linear ordering on the disk.

8. What is the drawback of contiguous allocation? How to overcome from it?

It suffers from external fragmentation. Compaction is used to solve the problem of external fragmentation. The original disk uses then freed completely by creating one large contiguous free space. Second problem with contiguous allocation algorithm is that with pre allocation. It is necessary to declare the size of the file at the time of creation.

9. What is linked allocation?

- Linked allocation solves all problems of contiguous allocation.
- Each block contains a pointer to the next block in the chain. The disk blocks may be scattered anywhere on the disk, The FAT contains a pointer to the first and last blocks of the file.
- To create a new file, simply create a new entry in FAT .With linked allocation each directory entry has a pointer to the first disk block of the file.

10. Write the disadvantages of Linked allocation

- The major problem is that it can be used effectively only for sequential access files. To find the *i*th block of a file, we must start at the beginning of that file and follow the pointers until we get to the *i*th block.

11. What is indexed allocation

- The FAT contains a separate one level index for each file, the index has one entry for each portion allocated to the file.
- The *i*th entry in the index block points to the *i*th block of the file.
- The directory contains the address of the index block.

12. Write about free space management

Since disk space is limited, we need to reuse the space from deleted files for new files, if possible. (Write-once optical disks only allow one write to any given sector, and thus such reuse is not physically possible.) To keep track of free disk space, the system maintains a free-space list. The free-space list records all *free* disk blocks—those not allocated to some file or directory.

13. How free list can be implemented?

1. Bit vector
2. Linked List
3. Grouping
4. Counting

14. Write about Bit vector

Frequently, the free-space list is implemented as a bit vector or bit map. Each block is represented by 1 bit. If the block is free, the bit is 1; if the block is allocated, the bit is 0.

15. Write about Linked List in free space management

Another approach to free-space management is to link together all the free disk blocks, keeping a pointer to the first free block in a special location on the disk and caching it in memory. This first block contains a pointer to the next free disk block, and so on.

16. Write about Grouping in free space management

A modification of the free-list approach stores the addresses of n free blocks in the first free block. The first $n-1$ of these blocks are actually free. The last block contains the addresses of another n free blocks, and so on. The addresses of a large number of free blocks can now be found quickly, unlike the situation when the standard linked-list approach is used.

17. Write about Counting in free space management

Rather than keeping a list of n free disk addresses, we can keep the address of the first free block and the number (n) of free contiguous blocks that follow the first block. Each entry in the free-space list then consists of a disk address and a count. Although each entry requires more space than would a simple disk address, the overall list is shorter, as long as the count is generally greater than 1.

18. What do you mean by seek time? (APR'12)(Nov 16) (Apr 17)

The total amount of time required for information on a disk drive to be found. The lower this value is the faster the hard drive will be able to find or read data. Examples of common hard drive **seeks times** are 8ms and 10ms.

19. List the disk performance parameter?(APR'13)

1. Seek Time: Seek time is the time required to move the disk arm to the required track.
2. Rotational Delay: Disk drive generally rotates at 3600 rpm, ie to make one revolution it takes around 16.7 ms. Thus on the average, the rotational delay will be 8.3 ms.
3. Transfer Time: The transfer time to or from the disk depends on the rotational speed of the disk

20. What is the purpose of device drivers? (APR'13)

A device driver is a file that lets the computer know the configuration and specifications of a certain hardware device. Some examples of devices that need drivers are hard drives, DVD drives, and PCI cards. Without the driver file, the computer will be unable to communicate with the device. Often, Windows will alert the user if there is a new device which doesn't have a driver installed yet. It will then hopefully let you install the correct driver for it.

21. List Different types of scheduling algorithm are as follows:

1. First come first serviced scheduling algorithm
2. Shortest seek time first algorithm
3. SCAN

4. Seek (or LOOK)
5. Circular scans(C-SCAN)
6. C-Seek (or C-LOOK)

22. Write about FCFS Scheduling

- FCFS-FIRST COME FIRST SERVE is the simplest form of disk scheduling.
- Service requests in the order they are received. This algorithm is similar to the FCFS job scheduling algorithm.
- FCFS is easy to implement but does not guarantee good throughput.

23. Write about SSTF Scheduling

- This algorithm selects the request with the minimum seek time from the current head position.
- SSTF may tend to move the head away from some requests during its local minimization.
- Seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position so always choosing the minimum seek time does not guarantee that the average seek time over a number of arm movements will be minimum. This choice should provide better performance than FCFS algorithm.

24. Write about Scan Scheduling:

- In this type of algorithm, the disk can start at one end of the disk and moves towards the other end.
- At the other end the direction of head movement is reversed and servicing continues.
- This algorithm is also otherwise called as elevator algorithm. before applying this algorithm, we should know the direction of head movement, in addition to the head current position.

25. Write about C-Scan Scheduling:

- C-SCAN SCHEDULING (CIRCULAR SCAN) is a variant of scan designed to provide a more uniform wait time.
 - When the head reaches the other end, however it immediately returns to the beginning of the disk, without servicing any request on the return trip.

26. Write about LOOK scheduling:

- The versions of C-SCAN and SCAN scheduling algorithm are called C-LOOK and LOOK scheduling.
- Both the algorithms move the disk across its full width. This is because they look for a request before continuing to move in a given direction.

27. Write about C-LOOK scheduling:

- This is a modification of seek algorithm, similar to C-SCAN. Here the arm goes only as far as the requests to be serviced and requests are serviced only in one direction.
- It is the best algorithm, since it involves minimum movement of the disk head to service all the requests. Disk head will not move, if there are no pending requests thus reducing wear and tear of the disk.

28. Write about Linux 1.0?

Linux 1.0 (March 1994) included these new features:

- Support for UNIX's standard TCP/IP networking protocols
 - BSD-compatible socket interface for networking programming
 - Device-driver support for running IP over an Ethernet
 - Enhanced file system
 - Support for a range of SCSI controllers for high-performance disk access
 - Extra hardware support
- Version 1.2 (March 1995) was the final PC-only Linux kernel

29. Write about Linux 2.0?

Linux 2.0

- Released in June 1996, 2.0 added two major new capabilities:
 - Support for multiple architectures, including a fully 64-bit native Alpha port. It supports multiprocessor architectures
- Other new features included:
 - Improved memory-management code
 - Improved TCP/IP performance
 - Support for internal kernel threads, for handling dependencies between loadable modules, and for automatic loading of modules on demand
 - Standardized configuration interface

- Available for Motorola 68000-series processors, Sun Spark systems, and for PC and PowerMac systems
- 2.4 and 2.6 increased SMP support, added journaling file system, preemptive kernel, 64-bit memory support

30. Define Slack ware?

The first Linux distributions managed these packages by simply providing a means of unpacking all the files into the appropriate places; modern distributions include advanced package management. Early distributions included SLS and **Slack ware**. **Slack ware** represents overall improvement in quality. **Red Hat** and **Debian** are popular distributions from commercial and noncommercial sources, respectively.

31. Write the Kernel modules are available in Linux?

- Sections of kernel code that can be compiled, loaded, and unloaded independent of the rest of the kernel
- A kernel module may typically implement a device driver, a file system, or a networking protocol
- The module interface allows third parties to write and distribute, on their own terms, device drivers or file systems that could not be distributed under the GPL
- Kernel modules allow a Linux system to be set up with a standard, minimal kernel, without any extra device drivers built in
- Three components to Linux module support:
 - **Module management** –allows modules to be loaded into memory.
 - **Driver registration** –allows modules to tell the rest of the kernel that a new driver has become available.
 - **Conflict resolution** –allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

32. Define Kernel in OS? (APR '12) (APR'15) (NOV '15)

A *kernel* is the part of the operating system that mediates access to system resources. It's responsible for enabling multiple applications to effectively share the hardware by controlling access to CPU, memory, disk I/O, and networking.

An *operating system* is the kernel plus applications that enable users to get something done (i.e. compiler, text editor, window manager, etc).

33. Define Module Management?

Supports loading modules into memory and letting them talk to the rest of the kernel. Module loading is split into two separate sections:

1. Managing sections of module code in kernel memory
2. Handling symbols that modules are allowed to reference

The module requestor manages loading requested, but currently unloaded, modules; it also regularly queries the kernel to see whether a dynamically loaded module is still in use, and will unload it when it is no longer actively needed

34. Define Driver Registration?

Allows modules to tell the rest of the kernel that a new driver has become available. The kernel maintains dynamic tables of all known drivers, and provides a set of routines to allow drivers to be added to or removed from these tables at any time. Registration tables include the following items:

1. **Device drivers**-These drivers include character devices (such as printers, terminals), block devices (including all disk drivers), and network interface devices.
2. **File systems** -It implements Linux's virtual -file -system calling routines.
3. **Network protocols**-It implements the entire networking protocol, such as IPX, or a new set of packet-filtering rules for a network firewall.
4. **Binary format**-Specifies a way of recognizing, and loading, a new type of executable file.

35. Define Conflict Resolution?

A mechanism that allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

The conflict resolution module aims to:

- Prevent modules from clashing over access to hardware resources
- Prevent **auto probes** from interfering with existing device drivers
- Resolve conflicts with multiple drivers trying to access the same hardware

36. Write the contents of Registration tables?

Registration tables include the following items:

1. **Device drivers**-These drivers include character devices (such as printers, terminals), block devices (including all disk drivers), and network interface devices.

2. **File systems** –It implements Linux's virtual –file –system calling routines.
3. **Network protocols**-It implements the entire networking protocol, such as IPX, or a new set of packet-filtering rules for a network firewall.
4. **Binary format**-Specifies a way of recognizing , and loading ,a new type of executable file.

37. What are the operations are available in process management?

UNIX process management separates the creation of processes and the running of a new program into two distinct operations.

1. The fork system call creates a new process
2. A new program is run after a call to execve

Under UNIX, a process encompasses all the information that the operating system must maintain to track the context of a single execution of a single program. Under Linux, process properties fall into three groups: the process's identity, environment, and context.

38. Define Scheduling Context?

The **scheduling context** is the most important part of the process context; it is the information that the scheduler needs to suspend and restart the process. The kernel maintains **accounting** information about the resources currently being consumed by each process, and the total resources consumed by the process in its lifetime so far.

39. Define signal-Handler table?

The **signal-handler table** defines the routine in the process's address space to be called when specific signals arrive. The **virtual-memory context** of a process describes the full contents of its private address space.

40. What is IPC? (APR'12)

Interprocess communication (IPC) is a set of programming interfaces that allow a programmer to coordinate activities among different program processes that can run concurrently in an operating system. This allows a program to handle many user requests at the same time. Since even a single user request may result in multiple processes running in the operating system on the user's behalf, the processes need to communicate with each other. The IPC interfaces make this possible. Each IPC method has its own advantages and limitations so it is not unusual for a single program to use all of the IPC methods.

41. Define file table?

The **file table** is an array of pointers to kernel file structures. When making file I/O system calls, processes refer to files by their index into this table. Whereas the file table lists the existing open files, the **file-system context** applies to requests to open new files. The current root and default directories to be used for new file searches are stored here.

42. Define Virtual Memory context?

The **virtual-memory context** of a process describes the full contents of its private address space.

43. Write any two advantages of Linux?(NOV'12)

1. **Open Source:** If you develop software that requires knowledge or modification of the operating system code, Linux's source code is at your fingertips. Most Linux applications are Open Source as well.
2. **Fast and easy installation:** Most Linux distributions come with user-friendly installation and setup programs. Popular Linux distributions come with tools that make installation of additional software very user friendly as well.
3. **Compatibility:** It runs all common Unix software packages and can process all common file formats.

44. Define static Linking?

A program whose necessary library functions are embedded directly in the program's executable binary file is **statically** linked to its libraries. The main disadvantage of static linkage is that every program generated must contain copies of exactly the same common system library functions.

45. Define Dynamic Linking?

Dynamic linking is more efficient in terms of both physical memory and disk-space usage because it loads the system libraries into memory only once.

46. Define Interprocess Communication?

Like UNIX, Linux informs processes that an event has occurred via signals. There is a limited number of signals, and they cannot carry information: Only the fact that a signal occurred is available to a process. The Linux kernel does not use signals to communicate with processes with

are running in kernel mode, rather, communication within the kernel is accomplished via scheduling states and wait ,queue structures.

47. Write any two design principles of Linux?

1. Linux is a multi-user, multitasking system with a full set of UNIX-compatible tools
 2. Its file system adheres to traditional UNIX semantics, and it fully implements the standard UNIX networking model
 3. Main design goals are speed, efficiency, and standardization
 4. Linux is designed to be compliant with the relevant POSIX documents; at least two Linux distributions have achieved official POSIX certification
- The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behavior

48. What is Process Identity?

Process ID (PID)- The unique identifier for the process; used to specify processes to the operating system when an application makes a system call to signal, modify, or wait for another process .

Credentials- Each process must have an associated user ID and one or more group IDs that determine the process's rights to access system resources and files

Personality- Not traditionally found on UNIX systems, but under Linux each process has an associated personality identifier that can slightly modify the semantics of certain system calls
Used primarily by emulation libraries to request that system calls be compatible with certain specific flavors of UNIX

49. Brief about Process Environment Block (PEB).(NOV'13)

When Windows Kernel loads a 64bit executable image in memory it builds automatically a PEB for it. The PEB (Process Environment Block) is process specific area of user land memory that

50. Write about fork() and clone() system call?

1. **fork** creates a new process with its own entirely new process context
2. **clone** creates a new process with its own identity, but that is allowed to share the data structures of its parent.

Using **clone** gives an application fine-grained control over exactly what is shared between two threads.

51. Does Linux supports Symmetric Multiprocessing?

Linux 2.0 was the first Linux kernel to support SMP hardware; separate processes or threads can execute in parallel on separate processors. To preserve the kernel's nonpreemptible synchronization requirements, SMP imposes the restriction, via a single kernel spin lock, that only one processor at a time may execute kernel-mode code

52. List the role of process manager?(NOV'13)

The Process Manager is responsible for creating new processes in the system and managing the most fundamental resources associated with a process. These services are all provided via messages. For example, if a running process wants to create a new process, it does so by sending a message containing the details of the new process to be created. Note that since messages are network-wide, you can easily create a process on another node by sending the process-creation message to the Process Manager on that node.

53. Write about Character Devices

A device driver which does not offer random access to fixed blocks of data. A character device driver must register a set of functions which implement the driver's various file I/O operations. The kernel performs almost no preprocessing of a file read or write request to a character device, but simply passes on the request to the device. The main exception to this rule is the special subset of character device drivers which implement terminal devices, for which the kernel maintains a standard interface

54. Write about few design principles of windows XP

1. Extensibility — layered architecture
 2. Portability —XP can be moved from on hardware architecture to another with relatively few changes
 3. Reliability —XP uses hardware protection for virtual memory, and software protection mechanisms for operating system resources
 4. Compatibility — applications that follow the IEEE 1003.1 (POSIX) standard can be complied to run on XP without changing the source code
- Performance —XP subsystems can communicate with one another via high-performance message passing.

55. Write about XP Architecture?

Layered system of modules

Protected mode — HAL, kernel, executive

User mode — collection of subsystems

Environmental subsystems emulate different operating systems

Protection subsystems provide security functions

56. Write about system components of XP?

Foundation for the executive and the subsystems

Never paged out of memory; execution is never preempted

Four main responsibilities: thread scheduling, interrupt and exception handling, low-level processor synchronization, recovery after a power failure.

Kernel is object-oriented, uses two sets of objects

Dispatcher objects :control dispatching and synchronization (events, mutants, mutexes, semaphores, threads and timers)

Control objects :(asynchronous procedure calls, interrupts, power notify, power status, process and profile objects)

57. Write about Kernel and Process and Threads?

The process has a virtual memory address space, information (such as a base priority), and an affinity for one or more processors. Threads are the unit of execution scheduled by the kernel's dispatcher. Each thread has its own state, including a priority, processor affinity, and accounting information. A thread can be one of six states: ready, standby, running, waiting, transition, and terminated.

58. What is Kernel Scheduling?

The dispatcher uses a 32-level priority scheme to determine the order of thread execution

Priorities are divided into two classes

1. The real-time class contains threads with priorities ranging from 16 to 31
2. The variable class contains threads having priorities from 0 to 15

59. What is soft real time?

Real-time threads are given preferential access to the CPU; but XP does not guarantee that a real-time thread will start to execute within any particular time limit. This is known as *soft real-time*.

60. Write about i/ o subsystem

The I/O manager is responsible for file systems,cache management ,device drivers,network drivers,Keeps track of which installable file systems are loaded, and manages buffers for I/O requests Works with VM Manager to provide memory-mapped file I/OControls the XP cache manager, which handles caching for the entire I/O systemSupports both synchronous and asynchronous operations, provides time outs for drivers, and has mechanisms for one driver to call another

61. What type of security is provided by Windows XP (APR'15)

The object-oriented nature of XP enables the use of a uniform mechanism to perform runtime access validation and audit checks for every entity in the system Whenever a process opens a handle to an object, the security reference monitor checks the process's security token and the object's access control list to see whether the process has the necessary rights. Security of an NTFS volume is derived from the XP object model .Each file object has a security descriptor attribute stored in this MFT record .This attribute contains the access token of the owner of the file, and an access control list that states the access privileges that are granted to each user that has access to the file

62. Write about file compression in XP

To compress a file, NTFS divides the file's data into *compression units*, which are blocks of 16 contiguous clusters. For sparse files, NTFS uses another technique to save space Clusters that contain all zeros are not actually allocated or stored on disk .Instead, gaps are left in the sequence of virtual cluster numbers stored in the MFT entry for the file. When reading a file, if a gap in the virtual cluster numbers is found, NTFS just zero-fills that portion of the caller's buffer

63. Write about memory management

A heap in the Win32 environment is a region of reserved address space

A Win 32 process is created with a 1 MB *default heap*

Access is synchronized to protect the heap's space allocation data structures from damage by concurrent updates by multiple threads

Because functions that rely on global or static data typically fail to work properly in a multithreaded environment, the thread-local storage mechanism allocates global storage on a per-thread basis. The mechanism provides both dynamic and static methods of creating thread-local storage

64. Write about interprocess communication in XP

Win32 applications can have interprocess communication by sharing kernel objects. An alternate means of interprocess communications is message passing, which is particularly popular for Windows GUI applications.

One thread sends a message to another thread or to a window.

A thread can also send data with the message.

Every Win32 thread has its own input queue from which the thread receives messages. This is more reliable than the shared input queue of 16-bit windows, because with separate queues, no stuck application can block input to the other applications.

65. What type of operating system is Windows XP? Describe two of its major features.

A 32/64 bit preemptive multitasking operating system supporting multiple users.

- i. The ability automatically to repair application and operating system problems.
- ii. Better networking and device experience (including digital photography and video).

66. List the design goals of Windows XP. Describe two in detail.

Design goals include security, reliability, Windows and POSIX application compatibility, high performance, extensibility, portability and international support.

- i. Reliability was perceived as a stringent requirement and included extensive driver verification, facilities for catching programming errors in user-level code, and a rigorous certification process for third-party drivers, applications, and devices.
- ii. Achieving high performance required examination of past problem areas such as I/O performance, server CPU bottlenecks, and the scalability of multithreaded and multiprocessor environments.

67. What are the responsibilities of the I/O manager?

The I/O manager is responsible for file systems, device drivers, and network drivers. The I/O manager keeps track of which device drivers, filter drivers, and file systems are loaded and manages buffers for I/O requests. It furthermore assists in providing memory mapped file I/O and controls the cache manager for the whole I/O system.

68. Sketch the components of a Linux system (NOV'14)

System management programs	User processes	User utility programs	compilers
System shared libraries			
Linux kernel			
Loadable kernel modules			

69. Define physical record and logical record. (NOV '15)

A physical record often is unstructured and has a fixed size related to the kind of physical media that stores it, and possibly to the location of the record on the media.

A logical record often is structured (has various program-specific fields) and might be stored in some number of full or partial physical records.

For example, a hard disk drive might be divided up into sectors (physical records) of 512 bytes each. If an application needs 1200 bytes of data for an address book entry (logical record) this will occupy portions of 3-4 sectors on disk.

70. Define latency time (Apr 17)

The rotational latency is the additional time for the disk to rotate the desired sector to the disk head.

71. What are the advantages of linked allocation? (Apr 17)

- There is no external fragmentation with linked allocation, and any free block on the free-space list can be used to satisfy a request.
- The size of a file need not be declared when that file is created.
- A file can continue to grow as long as free blocks are available.
- Consequently, it is never necessary to compact disk space.

72. What is swapping? What is the significant of swap space management? (Nov 16)

Swapping in that setting occurs when the amount of physical memory reaches a critically low point and processes (which are usually selected because they are the least active) are moved from memory to swap space to free available memory.

The main goal for the design and implementation of swap space is to provide the best throughput for the virtual memory system.

73. What is the use of boot block? (Apr 16)

A boot control block (per volume) can contain information needed by the system to boot an operating system from that volume.

74. What are the functions of Virtual File System (VFS)? (Apr 16)

A *virtual file system* (VFS) or *virtual file system* switch is an abstraction layer on top of a more concrete *file system*. The purpose of a VFS is to allow client applications to access different types of concrete *file systems* in a uniform way.

11 MARKS

1. Explain the allocation methods for disk space in file allocation? / Enumerate file allocation methods with example. (11) (NOV 12, 13) (APR '11)(APR'15) (Apr 16)(Nov 16)(Apr 17))(Nov 18)

Allocation methods:

Three major methods of allocating disk space are

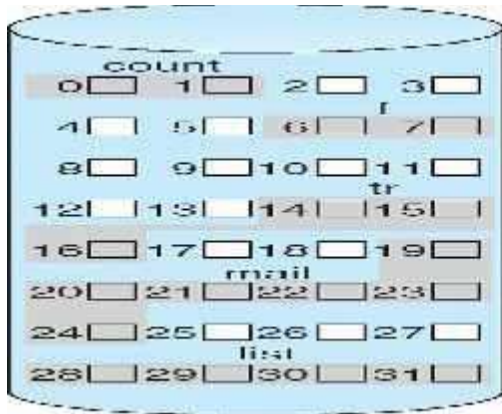
1. Contiguous allocation.
2. linked allocation.
3. indexed allocation.

Contiguous allocation

A single contiguous set of blocks is allocated to a file at the time of file creation; this is pre allocation strategy that uses portions of variable size. The FAT (File Allocation Table) needs just a single entry for each file, showing the starting block and the length of the file. Disk addresses define a linear ordering on the disk.

It suffers from external fragmentation. Compaction is used to solve the problem of external fragmentation. The original disk uses then freed completely by creating one large contiguous free space.

Second problem with contiguous allocation algorithm is that with pre allocation. It is necessary to declare the size of the file at the time of creation.

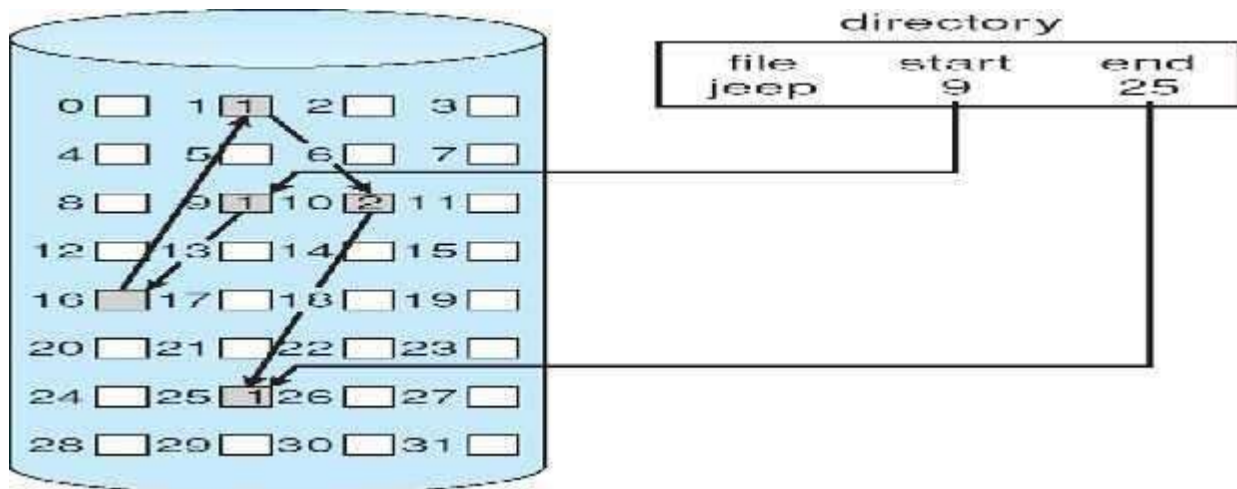


directory		
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Disadvantage:

- Dynamic allocation is not possible.
- The contiguous disk space allocation problem can be seen to be a particular application of the general dynamic storage allocation.
- Any management system can be used but some are slower than others.
- The other problem is external fragmentation because when it exist free space is broken into chunks.

Linked allocation:



- Linked allocation solves all problems of contiguous allocation.
- Each block contains a pointer to the next block in the chain. The disk blocks may be scattered anywhere on the disk, The FAT contains a pointer to the first and last blocks of the file.
- To create a new file, simply create a new entry in FAT .With linked allocation each directory entry has a pointer to the first disk block of the file.
- This pointer is initialized to nil to signify an empty file. Size field is also set to zero. There is no external fragmentation.

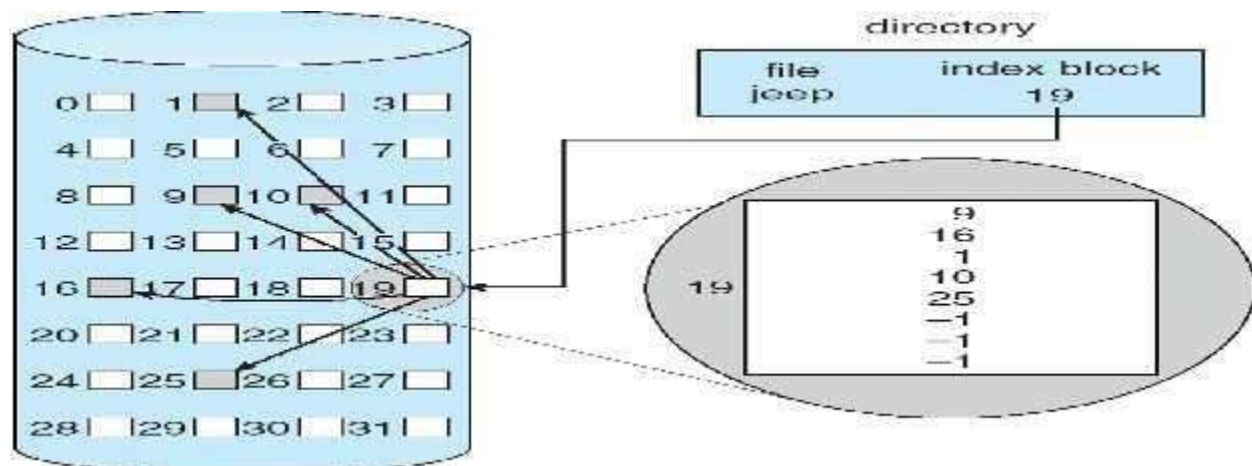
- The size of the file does not need to be declared when that file is created. A file can continue to grow as long as free blocks are available. It is never necessary to compact disk space.

Disadvantage:

The major problem is that it can be used effectively only for sequential access files. To find the *i*th block of a file, we must start at the beginning of that file and follow the pointers until we get to the *i*th block.

- Each access to a pointer requires a disk read and sometimes a disk seek
- Space is required for pointers.
- Pointers are used much smaller percentage of the file disk space.
- Other problem in linked allocation is reliability since the files are linked together by pointer scattered all over the disk.
- An important variation on linked allocation method is the use of a file allocation table(FAT).
- FAT is simple but efficient method of disk space allocation is used by MS-DOS and OS /2 operating systems.
- FAT is used same way as linked list.
- FAT allocation scheme can result in a significant number of disk head seeks, unless the FAT is cached.
- The disk head must move to the start of the volume to read the FAT.

Indexed allocation:



- The FAT contains a separate one level index for each file, the index has one entry for each portion allocated to the file.
- The *i*th entry in the index block points to the *i*th block of the file.
- The directory contains the address of the index block.
- Typically the file indexes are not physically stored as part of the FAT. Rather the file index for a file is kept in a separate block and the entry for the file in the FAT points to that block.

- Allocation may be on the basis of either fixed size block or variable size portions.
- When the file is created; all pointers in the index block are set to nil.
- When i^{th} block is first written, a block is obtained from the free space manager and its address is put in the i^{th} index block entry.
- Allocation by blocks eliminates external fragmentation, whereas allocation by variable size portions improves locality.
- Indexed allocation supports direct access and sequential access to the file and it is the most popular form of file allocation.
- Indexed allocation does not suffer from wasted space. The pointer overhead of the index block is generally greater than the pointer overhead of linked allocation.

2. Explain Disk Scheduling in detail with example? (APR '11) (APR '13) (NOV '15)

Disk Scheduling the amount of head movement needs to satisfy a series of I/O request can affect the performance. If the desired disk drive and controller are available; the request can be serviced immediately. If a device or controller is busy; any new requests for service will be placed on the queue of pending requests for that drive.

When one request is completed; the OS chooses which pending request to service next.

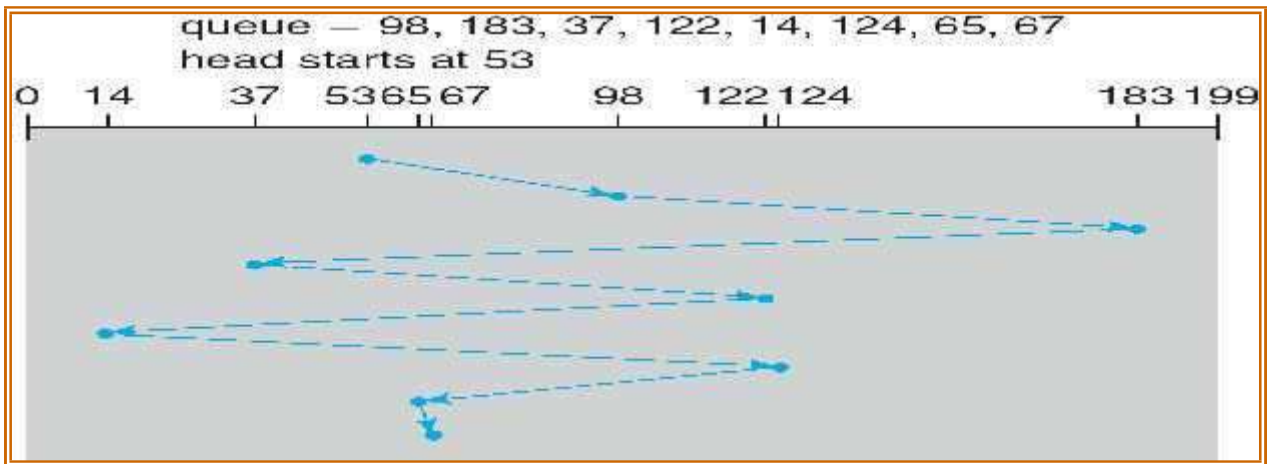
Different types of scheduling algorithm are as follows:

1. First come first serviced scheduling algorithm
2. Shortest seek time first algorithm
3. SCAN
4. Seek (or LOOK)
5. Circular scans(C-SCAN)
6. C-Seek (or C-LOOK)

FCFS Scheduling:

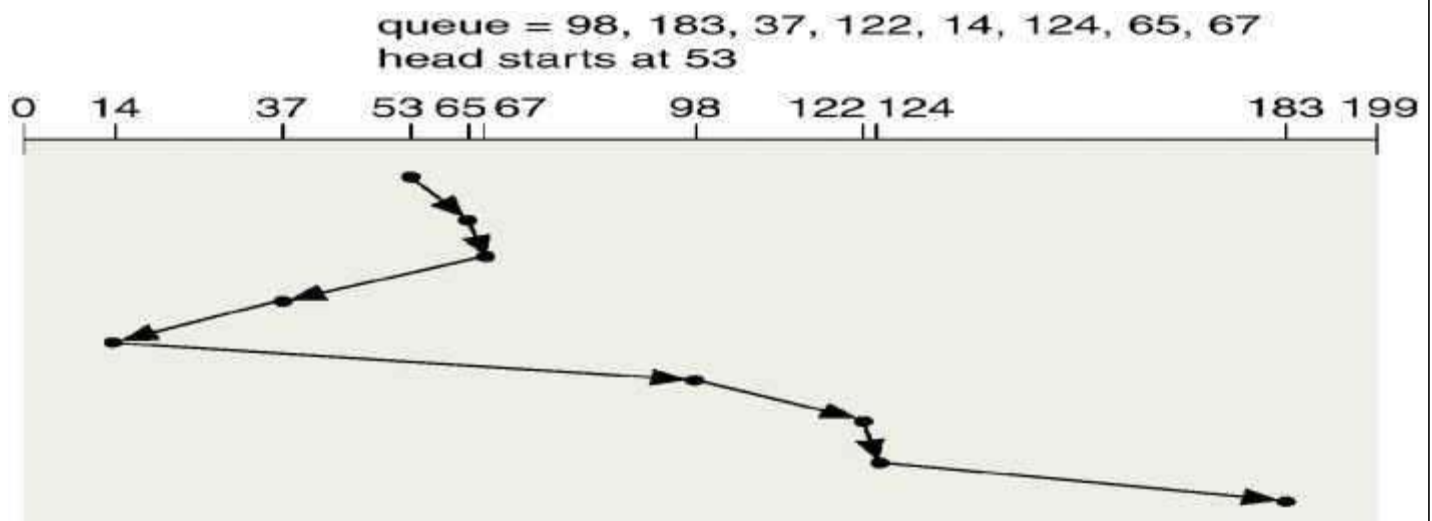
- FCFS-FIRST COME FIRST SERVE is the simplest form of disk scheduling.
- Service requests in the order they are received. This algorithm is similar to the FCFS job scheduling algorithm.
- FCFS is easy to implement but does not guarantee good throughput.
- It does not provide the fastest service.
- It takes no special action to minimize the overall seek time.
- This can be explained with an example as follows:

- Consider disk queue with requests for I/O to blocks in cylinders are, 98, 183, 37, 122, 14, 124, 65, 67.
- Here if the disk head is initially at 53, then it will move from 53 to 98 and then to this happens for a total head movements of 640 cylinders.
- Here the total head movements can be decreased simultaneously. suppose for cylinders 37 & 14 has to be serviced together, before or after the requests at 122 & 124, then the head movements can be decreased.



SSTF Scheduling:

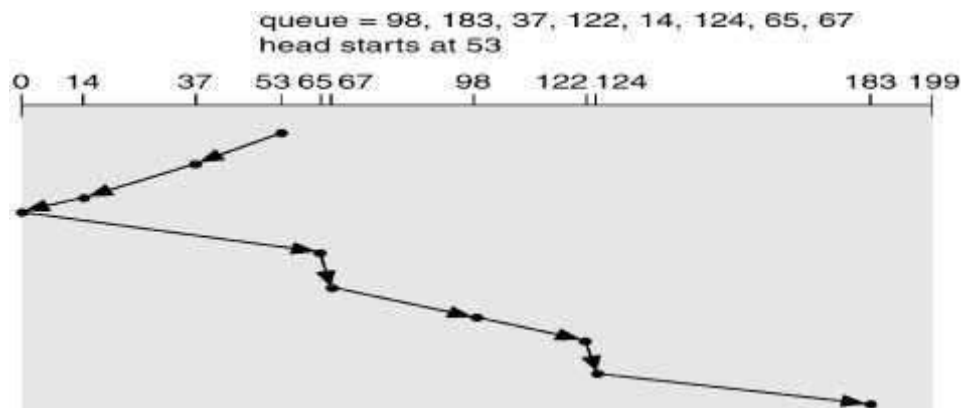
- This algorithm selects the request with the minimum seek time from the current head position.
- SSTF may tend to move the head away from some requests during its local minimization.
- Seek time increases with the number of cylinders traversed by the head, SSTF chooses the pending request closest to the current head position so always choosing the minimum seek time does not guarantee that the average seek time over a number of arm movements will be minimum. This choice should provide better performance than FCFS algorithm.



- From the above FCFS example, the closest request to the initial head position 53 is at cylinder 65. once we are at cylinder 65, the next closest request is at cylinder 67. from these, the request at cylinder 37 is closer than 98, so 37 is served next then finally we will reach 183.
- This scheduling method results in a total head movement of only 236 cylinders little more than more one-third of the distance needed for FCFS scheduling of this request queue.

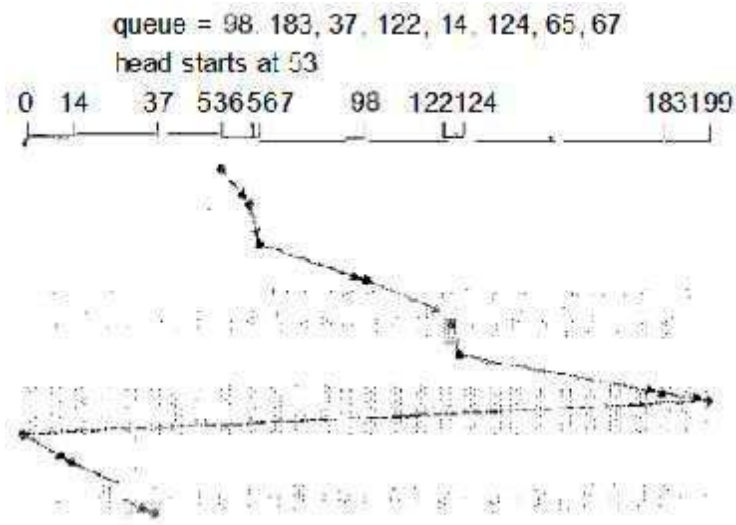
Scan Scheduling:

- In this type of algorithm, the disk can starts at one end of the disk and moves towards the other end.
- At the other end the direction of head movement is reversed and servicing continues.
- This algorithm is also otherwise called as elevator algorithm. before applying this algorithm, we should know the direction of head movement, in addition to the head current position.
- If the disk arm is moving towards 0, then the head will move towards the other end of the disk servicing the request at 65, 67, 98, 122, 124 & 183. first we have to service all
- The request going up, and then reversing it.



C-Scan Scheduling:

- C-SCAN SCHEDULING (CIRCULAR SCAN) is a variant of scan designed to provide a more uniform wait time.
 - When the head reaches the other end, however it immediately returns to the beginning of the disk, without servicing any request on the return trip.
 - It essentially treats the cylinder as a circular list.

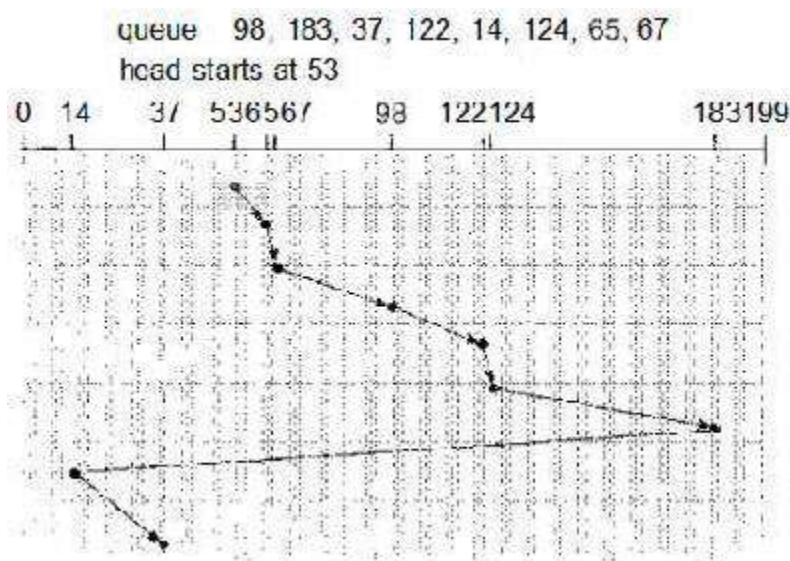


LOOK scheduling:

- The versions of C-SCAN and SCAN scheduling algorithm are called C-LOOK and LOOK scheduling.
- Both the algorithms move the disk across its full width. This is because they look for a request before continuing to move in a given direction.

C-LOOK scheduling:

- This is a modification of seek algorithm, similar to C-SCAN. Here the arm goes only as far as the requests to no serviced and requests are serviced only in one direction.
- It is the best algorithm, since it involves minimum movement of the disk head to service all the requests. Disk head will not move, if there is no pending requests thus reducing wear and tear of the disk.



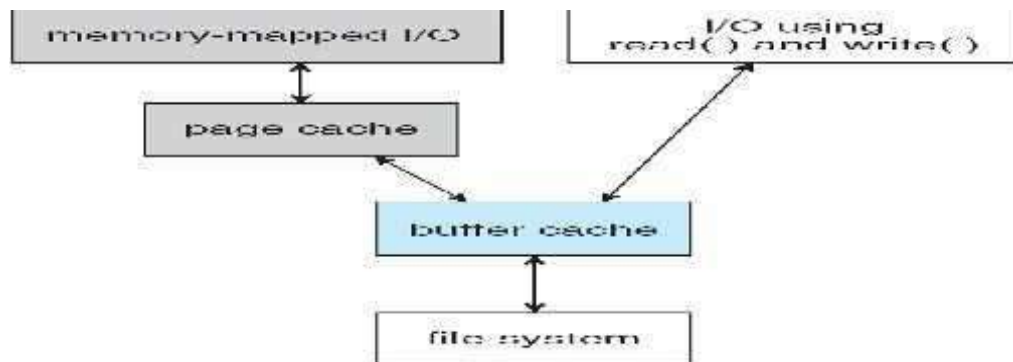
3. Discuss the various techniques used to improve the efficiency and performance of secondary storage? 6 Marks (APR '14)

Efficiency dependent on:

- Disk allocation and directory algorithms
- Types of data kept in file's directory entry
- Pre-allocation or as-needed allocation of metadata structures
- Fixed-size or varying-size data structures

In a Performance is Keeping data and metadata close together and Buffer cache – separate section of main memory for frequently used blocks ,Synchronous writes sometimes requested by apps or needed by OS, No buffering / caching – writes must hit disk before acknowledgement ,Asynchronous writes more common, buffer-able, faster Free-behind and read-ahead – techniques to optimize sequential access ,Reads frequently slower than writes.

A **page cache** caches pages rather than disk blocks using virtual memory techniques and addresses. Memory-mapped I/O uses a page cache. Routine I/O through the file system uses the buffer (disk) cache. This leads to the following figure



In an the above figure A **unified buffer cache** uses the same page cache to cache both memory-mapped pages and ordinary file system I/O to avoid **double caching**. **Consistency checking** is used to compares data in directory structure with data blocks on disk, and tries to fix inconsistencies. Can be slow and sometimes fails. Use system programs to **back up** data from disk to another storage device (magnetic tape, other magnetic disk, optical).Recover lost file or disk by **restoring** data from backup.

4. Briefly explain Disk management and swap space management? (NOV '12) (Apr 16)

Low level formatting or physical formatting: Dividing a disk into sectors that the disk controller can read and write. To use a disk to hold files, the operating system still needs to record its own data structures on the disk. Partition the disk into one or more groups of cylinders. Logical

formatting or “making a file system”. Boot block initializes system. The bootstrap is stored in ROM. Bootstrap loader program. Methods such as sector sparing used to handle bad blocks.

MS-DOS Disk Layout



Methods such as sector sparing (also known as forwarding) used to handle bad blocks. Spare sectors set aside on low-level formatting – Controller told to replace a bad sector logically with one of the spare sectors – To retain effectiveness of disk-scheduling optimization, provide spare sectors in each cylinder and also provides some spare cylinders. Use spare sector from same cylinder if possible.

Sector slipping is moves blocks following bad block downward (occupying spare sector) to free up block following bad Block skips bad block, using freed up block to hold that sector's information.

Swap-Space Management

Swap space Virtual memory uses disk space as an extension of main memory. Swap-space can be carved out of the normal file system, or, more commonly, it can be in a separate disk partition. Swap-space management allocates swap space 4.3BSD. When process starts, holds text segment (the program) and data segment. Kernel uses swap maps to track swap-space use. Solaris 2 allocates swap space only when a page is forced out of physical memory, not when the virtual memory page is first created. File data written to swap space until write to file system requested other dirty pages go to swap space due to no other home. Text segment pages thrown out and reread from the file system as needed

5. Explain in detail about Kernel architecture of Linux? (APR'13)(NOV'13))(Nov 18)

Linux is a modern, free operating system based on UNIX standards. First developed as a small but self-contained kernel in 1991 by Linux Torvalds, with the major design goal of UNIX compatibility. Its history has been one of collaboration by many users from all around the world, corresponding almost exclusively over the Internet. It has been designed to run efficiently and reliably on common PC hardware, but also runs on a variety of other platforms. The core Linux operating system kernel is entirely original, but it can run much existing free.

The Linux Kernel

Version 0.01 (May 1991) had no networking, ran only on 80386-compatible Intel processors and on PC hardware, had extremely limited device-drive support, and supported only the Minix file system

Linux Distributions:

Standard, precompiled sets of packages, or *distributions*, include the basic Linux system, system installation and management utilities, and ready-to-install packages of common UNIX tools. The first distributions managed these packages by simply providing a means of unpacking all the files into the appropriate places; modern distributions include advanced package management. Early distributions included SLS and **Slack ware**. **Slack ware** represents overall improvement in quality. **Red Hat and Debian** are popular distributions from commercial and noncommercial sources, respectively

The RPM Package file format permits compatibility among the various Linux distributions

Linux Licensing

- The Linux kernel is distributed under the GNU General Public License (GPL), the terms of which are set out by the Free Software Foundation
- Anyone using Linux, or creating their own derivative of Linux, may not make the derived product proprietary; software released under the GPL may not be redistributed as a binary-only product

Design Principles

1. Linux is a multi-user, multitasking system with a full set of UNIX-compatible tools
2. Its file system adheres to traditional UNIX semantics, and it fully implements the standard UNIX networking model
3. Main design goals are speed, efficiency, and standardization
4. Linux is designed to be compliant with the relevant POSIX documents; at least two Linux distributions have achieved official POSIX certification
5. The Linux programming interface adheres to the SVR4 UNIX semantics, rather than to BSD behavior

COMPONENTS OF A LINUX SYSTEM

Like most UNIX implementations, Linux is composed of three main bodies of code; the most important distinction between the kernel and all other components. The **kernel** is responsible for maintaining the important abstractions of the operating system. Kernel code executes in *kernel mode* with full access to all the physical resources of the computer. All kernel code and data structures are kept in the same single address space. The **system libraries** define a standard set of functions through which applications interact with the kernel, and which implement much of the operating-system functionality that does not need the full privileges of kernel code. The **system utilities** perform individual specialized management tasks. **Components of a Linux system shown below**

System management programs	User processes	User utility programs	compilers
System shared libraries			
Linux kernel			
Loadable kernel modules			

Kernel Modules

Sections of kernel code that can be compiled, loaded, and unloaded independent of the rest of the kernel

- A kernel module may typically implement a device driver, a file system, or a networking protocol
- The module interface allows third parties to write and distribute, on their own terms, device drivers or file systems that could not be distributed under the GPL
- Kernel modules allow a Linux system to be set up with a standard, minimal kernel, without any extra device drivers built in

Three components to Linux module support:

- **Module management** -allows modules to be loaded into memory.
- **Driver registration**-allows modules to tell the rest of the kernel that a new driver has become available.
- **Conflict resolution**-allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

Module Management

Supports loading modules into memory and letting them talk to the rest of the kernel. Module loading is split into two separate sections:

1. Managing sections of module code in kernel memory
2. Handling symbols that modules are allowed to reference

The module requestor manages loading requested, but currently unloaded, modules; it also regularly queries the kernel to see whether a dynamically loaded module is still in use, and will unload it when it is no longer actively needed

Driver Registration

Allows modules to tell the rest of the kernel that a new driver has become available the kernel maintains dynamic tables of all known drivers, and provides a set of routines to allow drivers to be added to or removed from these tables at any time. Registration tables include the following items:

Device drivers-

These drivers include character devices (such as printers, terminals) block devices (including all disk drivers), and network interface devices.

File systems –

It implements Linux's virtual –file –system calling routines.

Network protocols-

It implements the entire networking protocol, such as IPX, or a new set of packet-filtering rules for a network firewall.

Binary format-

Specifies a way of recognizing, and loading ,a new type of executable file.

Conflict Resolution

A mechanism that allows different device drivers to reserve hardware resources and to protect those resources from accidental use by another driver.

The conflict resolution module aims to:

- Prevent modules from clashing over access to hardware resources
- Prevent **auto probes** from interfering with existing device drivers
- Resolve conflicts with multiple drivers trying to access the same hardware

6. Explain in Detail about Process management? (NOV'14)

UNIX process management separates the creation of processes and the running of a new program into two distinct operations.

1. The fork system call creates a new process
2. A new program is run after a call to execute

Under UNIX, a process encompasses all the information that the operating system must maintain to track the context of a single execution of a single program. Under Linux, process properties fall into three groups: the process's identity, environment, and context Process Identity

Process ID (PID)-

The unique identifier for the process; used to specify processes to the operating system when an application makes a system call to signal, modify, or wait for another process

Credentials-

Each process must have an associated user ID and one or more group IDs that determine the process's rights to access system resources and files

Personality-

Not traditionally found on UNIX systems, but under Linux each process has an associated personality identifier that can slightly modify the semantics of certain system calls

Used primarily by emulation libraries to request that system calls be compatible with certain specific flavors of UNIX

Process Environment

The process's environment is inherited from its parent, and is composed of two null-terminated vectors:

1. The argument vector lists the command-line arguments used to invoke the running program; conventionally starts with the name of the program itself
2. The environment vector is a list of "NAME=VALUE" pairs that associates named environment variables with arbitrary textual values

Passing environment variables among processes and inheriting variables by a process's children are flexible means of passing information to components of the user-mode system software. The environment-variable mechanism provides a customization of the operating system that can be set on a per-process basis, rather than being configured for the system as a whole

Process Context

- The (constantly changing) state of a running program at any point in time
- The **scheduling context** is the most important part of the process context; it is the information that the scheduler needs to suspend and restart the process.
- The kernel maintains **accounting** information about the resources currently being consumed by each process, and the total resources consumed by the process in its lifetime so far.
- The **file table** is an array of pointers to kernel file structures. When making file I/O system calls, processes refer to files by their index into this table. Whereas the file table lists the existing open files, the **file-system context** applies to requests to open new files. The current root and default directories to be used for new file searches are stored here.
- The **signal-handler table** defines the routine in the process's address space to be called when specific signals arrive. The **virtual-memory context** of a process describes the full contents of the private address space.

Processes and Threads

Linux uses the same internal representation for processes and threads; a thread is simply a new process that happens to share the same address space as its parent .A distinction is only made when a new thread is created by the **clone** system call.

- **fork** creates a new process with its own entirely new process context
- **Clone** creates a new process with its own identity, but that is allowed to share the data structures of its parent.

Using **clone** gives an application fine-grained control over exactly what is shared between two threads.

7. Explain in detail about Process Scheduling?

Linux uses two process-scheduling algorithms:

- 1.A time-sharing algorithm for fair preemptive scheduling between multiple processes
2. A real-time algorithm for tasks where absolute priorities are more important than fairness

A process's scheduling class defines which algorithm to apply .For time-sharing processes; Linux uses a prioritized, credit based algorithm. The crediting rule factors in both the process's history and its priority This crediting system automatically prioritizes interactive or I/O-bound processes. Linux implements the FIFO and round-robin real-time scheduling classes; in both cases, each process has a priority in addition to its scheduling class.

The scheduler runs the process with the highest priority; for equal-priority processes, it runs the process waiting the longest. FIFO processes continue to run until they either exit or block .A round-robin process will be preempted after a while and moved to the end of the scheduling queue, so that round-robin processes of equal priority automatically time-share between themselves.

Symmetric Multiprocessing

Linux 2.0 was the first Linux kernel to support SMP hardware; separate processes or threads can execute in parallel on separate processors. To preserve the kernel's nonpreemptible synchronization requirements, SMP imposes the restriction, via a single kernel spin lock, that only one processor at a time may execute kernel-mode code

The job of allocating CPU time to different tasks within an operating system. While scheduling is normally thought of as the running and interrupting of processes, in Linux, scheduling also includes the running of the various kernel tasks .Running kernel tasks encompasses both tasks that are requested by a running process and tasks that execute internally on behalf of a device driver. new scheduling algorithm – preemptive, priority-based

- Real-time range
- Nice value

Kernel Synchronization

A request for kernel-mode execution can occur in two ways:

1. A running program may request an operating system service, either explicitly via a system call, or implicitly, for example, when a page fault occurs
2. A device driver may deliver a hardware interrupt that causes the CPU to start executing a kernel-defined handler for that interrupt.

Kernel synchronization requires a framework that will allow the kernel's critical sections to run without interruption by another critical section.

Linux uses two techniques to protect critical sections:

1. Normal kernel code is nonpreemptible when a time interrupt is received while a process is executing a kernel system service routine, the kernel's need reached flag is set so that the scheduler will run once the system call has completed and control is about to be returned to user mode.
2. The second technique applies to critical sections that occur in an interrupt service routine. By using the processor's interrupt control hardware to disable interrupts during a critical section, the kernel guarantees that it can proceed without the risk of concurrent access of shared data structures.

To avoid performance penalties, Linux's kernel uses a synchronization architecture that allows long critical sections to run without having interrupts disabled for the critical section's entire duration. Interrupt service routines are separated into a *top half* and a *bottom half*.

The top half is a normal interrupt service routine, and runs with recursive interrupts disabled. The bottom half is run, with all interrupts enabled, by a miniature scheduler. That ensures that bottom halves never interrupt themselves this architecture is completed by a mechanism for disabling selected bottom halves while executing normal, foreground kernel code.

Interrupt Protection Levels

Top-half interrupt handlers
Bottom-half interrupt handlers
Kernel system service routines (preemptible)
User mode programs (preemptible)

Fig 5.2 Interrupt protocol Level

Each level may be interrupted by code running at a higher level, but will never be interrupted by code running at the same or a lower level. User processes can always be preempted by another process when a time-sharing scheduling interrupt occurs.

8. Explain how interprocess communication is carried out in Linux OS? (NOV'12) (APR'13) (APR'15)

Interprocess Communication

Many operating systems provide mechanisms for interprocess communication (IPC)

Processes must communicate with one another in multiprogrammed and networked environments

For example, a Web browser retrieving data from a distant server

Essential for processes that must coordinate activities to achieve a common goal.

Signals

Software interrupts that notify a process that an event has occurred. Do not allow processes to specify data to exchange with other processes. Processes may catch, ignore or mask a signal

- Catching a signal involves specifying a routine that the OS calls when it delivers the signal
- Ignoring a signal relies on the operating system's default action to handle the signal
- Masking a signal instructs the OS to not deliver signals of that type until the process clears the signal mask

Message Passing

Message-based interprocess communication Messages can be passed in one direction at a time .One process is the sender and the other is the receiver Message passing can be bidirectional Each process can act as either a sender or a receiver. Messages can be blocking or nonblocking Blocking requires the receiver to notify the sender when the message is received.Nonblocking enables the sender to continue with other processing. Popular implementation is a pipe A region of memory protected by the OS that serves as a buffer, allowing two or more processes to exchange data.

LUNIX Processes

- LUNIX processes

All processes are provided with a set of memory addresses, called a virtual address space

A process's PCB is maintained by the kernel in a protected region of memory that user processes cannot access

A LUNIX PCB stores:

- The contents of the processor registers

- PID
- The program counter
- The system stack

All processes are listed in the process table

9. Explain in detail about Linux advantage?

Linux Advantages

1. **Low cost:** You don't need to spend time and money to obtain licenses since Linux and much of its software come with the GNU General Public License. You can start to work immediately without worrying that your software may stop working anytime because the free trial version expires. Additionally, there are large repositories from which you can freely download high quality software for almost any task you can think of.
2. **Stability:** Linux doesn't need to be rebooted periodically to maintain performance levels. It doesn't freeze up or slow down over time due to memory leaks and such. Continuous up-times of hundreds of days (up to a year or more) are not uncommon.
3. **Performance:** Linux provides persistent high performance on workstations and on networks. It can handle unusually large numbers of users simultaneously, and can make old computers sufficiently responsive to be useful again.
4. **Network friendliness:** Linux was developed by a group of programmers over the Internet and has therefore strong support for network functionality; client and server systems can be easily set up on any computer running Linux. It can perform tasks such as network backups faster and more reliably than alternative systems.
5. **Flexibility:** Linux can be used for high performance server applications, desktop applications, and embedded systems. You can save disk space by only installing the components needed for a particular use. You can restrict the use of specific computers by installing for example only selected office applications instead of the whole suite.
6. **Compatibility:** It runs all common Unix software packages and can process all common file formats.
7. **Choice:** The large number of Linux distributions gives you a choice. Each distribution is developed and supported by a different organization. You can pick the one you like best; the core functionalities are the same; most software runs on most distributions.
8. **Fast and easy installation:** Most Linux distributions come with user-friendly installation and setup programs. Popular Linux distributions come with tools that make installation of additional software very user friendly as well.

9. Full use of hard disk: Linux continues work well even when the hard disk is almost full.
10. Multitasking: Linux is designed to do many things at the same time; e.g., a large printing job in the background won't slow down your other work.
11. Security: Linux is one of the most secure operating systems. "Walls" and flexible file access permission systems prevent access by unwanted visitors or viruses. Linux users have to option to select and safely download software, free of charge, from online repositories containing thousands of high quality packages. No purchase transactions requiring credit card numbers or other sensitive personal information are necessary.
12. Open Source: If you develop software that requires knowledge or modification of the operating system code, Linux's source code is at your fingertips. Most Linux applications are Open Source as well.

10. Explain how the memory and file management is implemented in windows XP?

(APR'13)(Nov 18)

Memory Management

Virtual memory manager (VMM):

Executive component responsible for managing memory. Lazy allocation: Avoid allocating memory until necessary. Perfecting: Move pages from disk to main memory before they are needed

Page file: Stores pages that do not fit in main memory, Windows XP supports up to 16 page files

Memory Organization

In an 2-bit virtual address space consists of Windows 64-Bit Edition has 64-bit address space. 4GB virtual address space per process. Process can access only user space.VMM stores page tables and other data in system space 2GB user space, 2GB system space, 4KB pages. In an 2-level hierarchical memory map consists of Page directory table, page table, page frame. It includes Page directory entries (PDEs) point to page table consists of one page directory table per process, Location in page directory register, Page table- Page table entries (PTEs) point to page frames Page frame Contains page of data.

Windows File system:

Windows supports a number of file systems, including the file allocation table (FAT) that runs on Windows 95, MS-DOS, and OS/2. But the developers of Windows also designed a new fileSystem, the Windows File System (NTFS), that is intended to meet high-end requirements forWorkstations and servers. Examples of high-end applications: Client/server applications such as file servers, compute servers, and database servers, Resource-intensive engineering and scientific applications. Network applications for large corporate systems .NTFS is a flexible and powerful file system built, as

which shall see, on an elegantly simple file system model. The most noteworthy features of NTFS include: Recoverability:

High on the list of requirements for the new Windows file system was the ability to recover from system crashes and disk failures. In the event of such failures, NTFS is able to reconstruct disk volumes and return them to a consistent state. It does this by using a transaction processing model for changes to the file system; each significant change is treated as an atomic action that is either entirely performed or not performed at all. Each transaction that was in process at the time of a failure is subsequently backed out or brought to completion. In addition, NTFS uses redundant storage for critical file system data, so that failure of a disk sector does not cause the loss of data describing the structure and status of the file system.

Security:

NTFS uses the Windows object model to enforce security. An open file is implemented as a file object with a security descriptor that defines its security attributes.

Large disks and large files:

NTFS supports very large disks and very large files more efficiently than most other file systems, including FAT.

Multiple data streams:

The actual contents of a file are treated as a stream of bytes. In NTFS it is possible to define multiple data streams for a single file. An example of the utility of this feature is that it allows Windows to be used by remote Macintosh systems to store and retrieve files. On Macintosh, each file has two components: the file data and a resource fork that contains information about the file. NTFS treats these two components as two data streams.

General indexing facility:

NTFS associates a collection of attributes with each file. The set of file descriptions in the file management system is organized as a relational database, so that files can be indexed by any attribute.

11. Explain in detail about Security? (NOV'14)

The *pluggable authentication modules (PAM)* system is available under Linux. PAM is based on a shared library that can be used by any system component that needs to authenticate users. Access control under UNIX systems, including Linux, is performed through the use of unique numeric identifiers (**uid** and **gid**). Access control is performed by assigning objects a *protections mask*, which specifies which access modes—read, write, or execute—are to be granted to processes with owner, group, or world access.

Linux augments the standard UNIX **setid** mechanism in two ways:

It implements the POSIX specification's saved *user-id* mechanism, which allows a process to repeatedly drop and reacquire its effective uid. It has added a process characteristic that grants just a subset of the rights of the effective uid. Linux provides another mechanism that allows a client to selectively pass access to a single file to some server process without granting it any other privileges.

12. Explain in detail about Windows XP System Architecture?(NOV'12) (APR'15)

The architecture of windows xp consist of 2 modes

Protected mode

User mode

The main layers of protected mode are hardware abstraction layer, kernel and executive.

The collection of subsystem and services are called as user mode.

User mode subsystems fall into two categories.

Environment subsystem – which emulate different operating system

Protection subsystem – which provide security function

Protected mode:

1. Hardware abstraction layer :

The HAL is the layer of software that hides hardware differences from upper levels of the operating system to help make Windows XP portable.

The HAL exports a virtual machine interface that is used by the kernel dispatcher , executive and the drivers.

The advantage of this approach is that only a single version of each device driver is required.

The HAL also provide support for symmetric multiprocessing.

2. Kernel:

The kernel of windows xp provides the foundations for the executive and the subsystems.

The kernel remains in memory and its execution is never preempted. It has 4 responsibilities.

They are thread scheduling, interrupt and execution handling, low-level processor synchronization and recovery after a power failure.

The kernel is object oriented. Kernel has kernel dispatcher.

Kernel dispatcher:

Kernel dispatcher provides the foundation for the executive and subsystems.

The dispatcher is never paged out of memory and its execution is never preempted.

Its main responsibilities are thread scheduling , implementation of synchronization primitives , software interrupts and execution dispatching.

➤ Thread scheduling:

Windows xp uses the concepts of processor and thread for executable code.

The process has a virtual memory address and information used to initialize each thread.

Each process has one or more thread each of which is an executable unit dispatched by the kernel.

There are 6 possible thread states. They are

Ready – indicates waiting to run

Standby – highest priority ready thread is moved to standby state

Running –executing on a processor

Waiting –waiting for an dispatcher object

Transition –a new thread is in transition state while its waits for resources necessary for execution.

Termination –finishes execution.

➤ Implementation of synchronization primitives:

The os data structures are managed as object using common facilities for allocation, reference counting and security.

➤ Software interrupt:

2 types of software interrupt

- Asynchronous procedure call:

It is used to begin execution of a new thread, terminate processes and to deliver notification that an asynchronous I/O has completed.

- Deferred procedure call:

It is used to postpone interrupt processing.

➤ Exception and interrupts:

The Windows XP has several exception including

1. Memory access violation
2. Integer overflow
3. Floating point overflow or underflow
4. Integer divide by zero
5. Floating point divide by zero etc.

This exception are handled by the exception dispatcher when an exception occurs in kernel mode, the exception dispatcher simply calls a routine to locate the exception handler.

3. Executive :

The Windows XP executive provides a set of services that all environment subsystems use.

The services are grouped as follows:

- Object manager ,
- virtual memory manager ,
- process manager ,
- local procedure call facility ,
- I/O manager,
- security references monitor ,
- plug and play and security manager ,
- register and booting.

➤ Object manager

Windows XP uses a generic set of interface s for managing the kernel entities that is manipulating by user mode program.

Windows XP calls these entities objects and the executive component that manipulate them is the object manager.

The job of the object manager is to supervise the use of all the managed objects.

When a thread wants to use an object, it calls the object managers open method to get a reference to the object.

➤ Virtual memory manager:

Then executive component that manages the virtual address space, physical memory allocation and paging is the virtual memory manager.

The design of VM manager assumes that the underlying hardware support virtual to physical mapping, a paging mechanism , transparent cache coherence on multiprocessor system and allow multiple page table entries to map to the same physical page frame.

The VM manager for IA32 processor has a page directory that contains 1024 page directory entries (PDE) of size 4 bytes. Each PTE point to a 4 kb page frames.

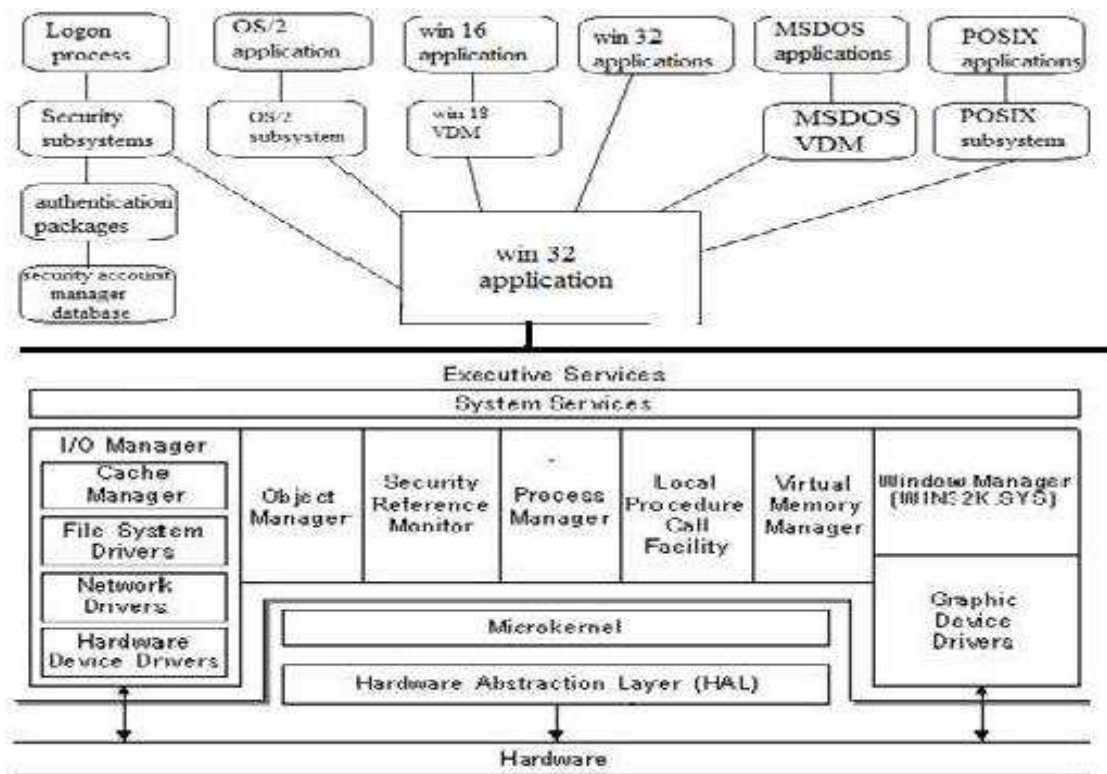


Fig 5.3 Block Diagram of Windows XP

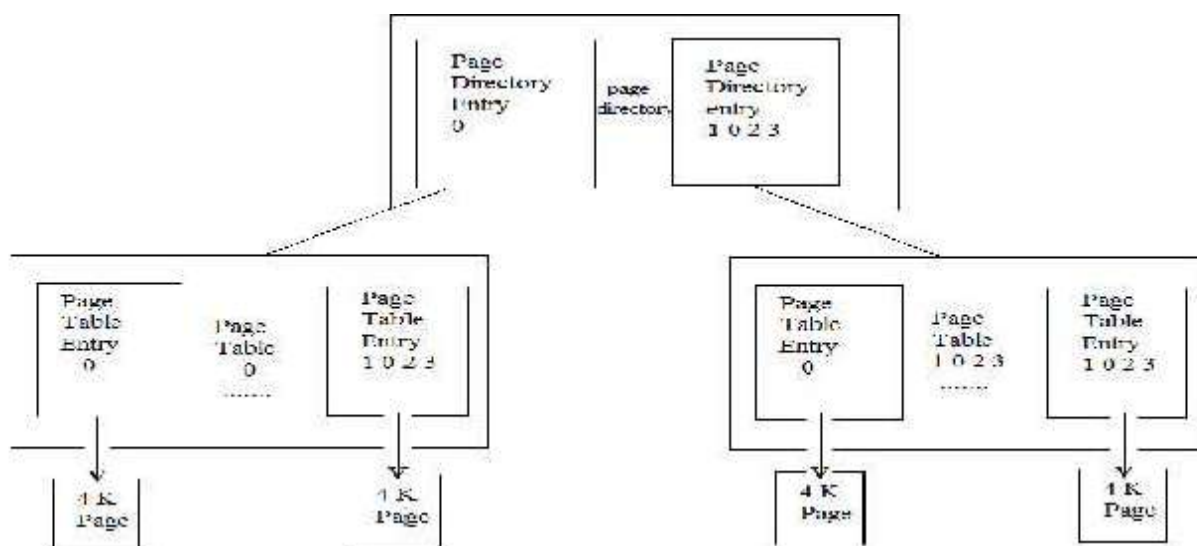
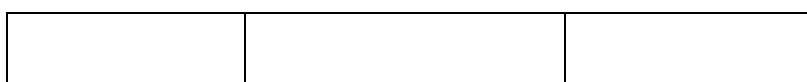


Fig 5.4 PTE point to a 4 kb page frame.

A 32 bit virtual memory address is splitted into 3 value

- First 10 bit – used as an index into page directory
- Next 10 bit – used to select a file from page table
- The remaining 12 bits are the offset of the specific in the page frame.



PDE	PTE	Page offset
-----	-----	-------------

Fig 5.5 Page Directory

➤ Process manager:

The windows xp process manager provides services for creating, deleting and using processes, threads and jobs.

It has no knowledge about parent child relationship or process hierarchies.

The pro manager is also not involved in the scheduling of processes other than setting the priorities

And the affirmatives in processes and threads when they are created.

Thread scheduling takes place in kernel dispatcher.

➤ Local procedure call facility:

The implementation of Windows XP uses a client server model.

The OS uses local procedure call (LPC) to pass request and result between client and server process within a single machine.

The various Windows XP subsystems.

When an LPC channel is created, one of three message passing techniques must be specified.

1. The first tech is suitable for small messages. In this parts message queue is used as intermediate storage, the message are copied from one process to the other.
2. The second tech is for large messages. Message send through the ports message queue contain a pointer and size information referring to the section object.
This avoids the need to copy large messages. The sender places data into the shared section and receiver views them directly.
3. The third tech of LPC message passing uses the API that read / writes directly into a process address space.

➤ I/O manager :

I/O manager is responsible for file system, device drivers and network drivers.

It keeps track of which device drivers, filter drivers and file systems are loaded and also manages buffer for I/O request.

Device driver are arranged as a list for each the I/O manager convert the request it receive into standard form called as I/O request packet (IRP)

➤ Cache manager:

In many OS, caching is done by the file system. The cache manager works closely with the VM manager to provide cache services for all components under the control of I/O manager.

➤ **Security reference monitor:**

The security reference monitor (SRM) is also responsible for manipulating the privilege in security tokens.

Whenever a process opens a handle to an object the security reference monitor (SRM) checks the process.

➤ **Plug and play and power manager:**

The OS uses the Plug and play manager to recognize and adapt the change in the hardware configuration.

Windows XP also moves to system to a state requiring lower power consumption.

2. User mode:

1. Environmental subsystem:

It is a user modes processes layered over the native windows xp executive services to enable windows xp to run program developed for other os including 16 bit windows , MS DOS and POSIX.

- **MS DOS environment:**

The MS-DOS environment does not have of other windows xp environment subsysytems.

It is provided by a win 32 application called the virtual DOS machine (VDM)

- **16-bit windows environment:**

The win16 execution environment is provided by a VDM that incorporates additional software called windows on windows that provides the windows 3.1 kernel routines and stub routine for window manager and graphical device interface functions.

POSIX subsystem:

The POSIX system is designed to run POSIX application written to follow the POSIX standard, which is based on the UNIX model.

POSIX applications can be started by the win 32 sub system or by another POSIX application.

2. Logon and security subsystems:

Before a user can access objects on windows xp, that user must be authenticated by the logon service, win logon.

It is responsible for responding to secure attention sequence.

The local security authority subsystem (LSASS) is the process that generates access tokens to represent users on the subsystems.

It calls an authentication package to perform authentication using information from the log on subsystem or network server.

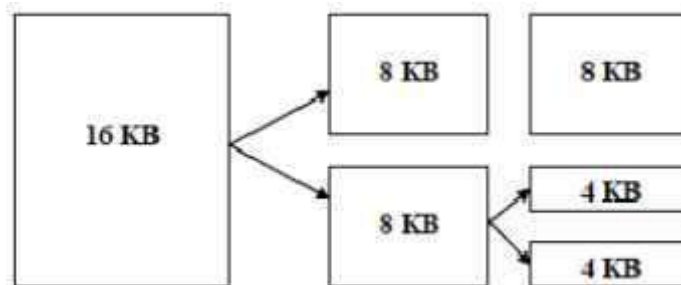
The security subsystem then generates the access token for the user ID containing the appropriate privileges , quota limits and group IDS.

The default authentication package for windows xp domains is Kerberos.

13. Describe about memory allocation stages. (11)(NOV'13)

Linux's physical memory-management system deals with allocating and freeing pages, groups of pages, and small blocks of memory. It has additional mechanisms for handling virtual memory, memory mapped into the address space of running processes. Splits memory into 3 different zones due to hardware characteristics.

Splitting of Memory in a Buddy Heap



Managing Physical Memory

The page allocator allocates and frees all physical pages; it can allocate ranges of physically-contiguous pages on request. The allocator uses a buddy-heap algorithm to keep track of available physical pages. Each allocatable memory region is paired with an adjacent partner. Whenever two allocated partner regions are both freed up they are combined to form a larger region. If a small memory request cannot be satisfied by allocating an existing small free region, then a larger free region will be subdivided into two partners to satisfy the request. Memory allocations in the Linux kernel occur either statically (drivers reserve a contiguous area of memory during system boot time) or dynamically (via the page allocation). Also uses slab allocate for kernel memory.

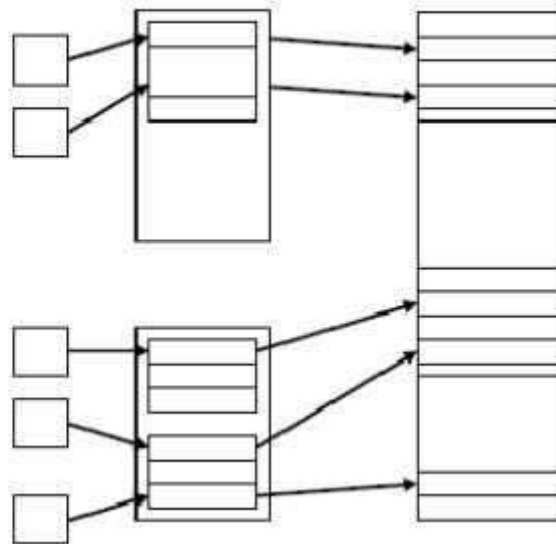
Kernel objects

caches

slabs

3 KB
objects

7 KB
objects



Virtual Memory

The VM system maintains the address space visible to each process: It creates pages of virtual memory on demand, and manages the loading of those pages from disk or their swapping back out to disk as required.

The VM manager maintains two separate views of a process's address space: A logical view describing instructions concerning the layout of the address space.

The address space consists of a set of no overlapping regions, each representing a continuous, page-aligned subset of the address space. A physical view of each address space which is stored in the hardware page tables for the process. Virtual memory regions are characterized by:

1. The backing store, which describes from where the pages for a region come; regions are usually backed by a file or by nothing (*demand-zero* memory).
2. The region's reaction to writes (page sharing or copy-on-write)

The kernel creates a new virtual address space. When a process runs a new program with the **exec** system call. Upon creation of a new process by the **fork** system call. On executing a new program, the process is given a new, completely empty virtual-address space; the program-loading routines populate the address space with virtual-memory regions.

Creating a new process with **fork** involves creating a complete copy of the existing process's virtual address space. The kernel copies the parent process's VMA descriptors, then creates a new set of page tables for the child. The parent's page tables are copied directly into the child's, with the reference count of each page covered being incremented. After the fork, the parent and child share the same physical pages of memory in their address spaces. The VM paging system

relocates pages of memory from physical memory out to disk when the memory is needed for something else. The VM paging system can be divided into two sections:

1. The page out-policy algorithm decides which pages to write out to disk, and when
2. When, the paging mechanism actually carries out the transfer, and pages data back into physical memory as needed. The Linux kernel reserves a constant, architecture-dependent region of the virtual address space of every process for its own internal use .

This kernel virtual-memory area contains two regions:

1. A static area that contains page table references to every available physical page of memory in the system, so that there is a simple translation from physical to virtual addresses when running kernel code
2. The reminder of the reserved section is not reserved for any specific purpose; its page table entries can be modified to point to any other areas of memory.

Executing and Loading User Programs

Linux maintains a table of functions for loading programs; it gives each function the opportunity to try loading the given file when an exec system call is made. The registration of multiple loader routines allows Linux to support both the ELF and a. Outbinary formats. Initially, binary-file pages are mapped into virtual memory .

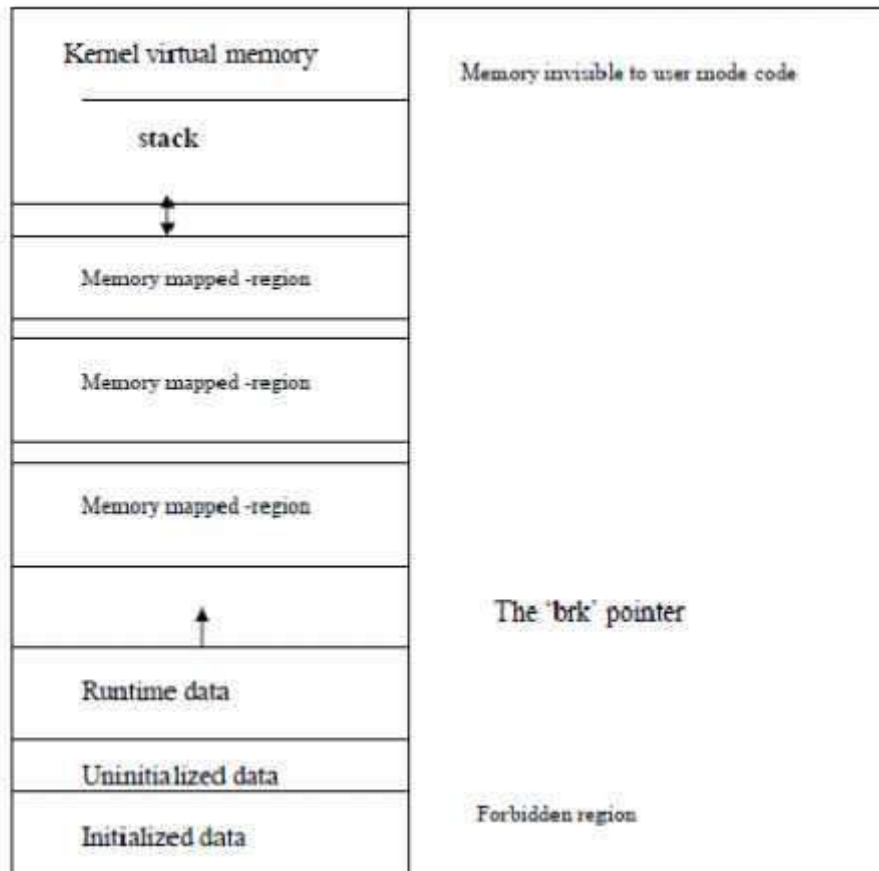
Only when a program tries to access a given page will a page fault result in that page being loaded into physical memory. An ELF-format binary file consists of a header followed by several page-aligned sections. The ELF loader works by reading the header and mapping the sections of the file into separate regions of virtual memory

Static and Dynamic Linking

A program whose necessary library functions are embedded directly in the program's executable binary file is **statically** linked to its libraries. The main disadvantage of static linkage is that very program generated must contain copies of exactly the same common system library functions.

Dynamic linking is more efficient in terms of both physical memory and disk-space usage because it loads the system libraries into memory only once.

Memory Layout for ELF Programs



14. Explain how the following mechanisms are implemented in Windows XP?

(1) Thread (2) IPC (APR'12)

Thread:

The idea is to have separate threads of control (hence the name) running in the same address space. An *address space* is a memory management concept. For now think of an address space as the memory in which a process runs and the mapping from the virtual addresses (addresses in the program) to the physical addresses (addresses in the machine). Each thread is somewhat like a process (e.g., it is scheduled to run) but contains less state (e.g., the address space belongs to the process in which the thread runs).

The Thread Model:

A process contains a number of resources such as address space, open files, accounting information, etc. In addition to these resources, a process has a thread of control, e.g., program counter, register contents, stack. The idea of threads is to permit multiple threads of control to execute within one process. This is often called **multithreading** and threads are often called **lightweight processes**. Because threads in the same process share so much state, switching between them is much less expensive than switching between separate processes.

Individual threads within the same process are not completely independent. For example there is no memory protection between them. This is typically not a security problem as the threads are cooperating and all are from the same user (indeed the same process). However, the shared resources do make debugging harder. For example one thread can easily overwrite data needed by another and if one thread closes a file other threads can't read from it.

Implementing Threads in the Kernel:

Move the thread operations into the operating system itself. This naturally requires that the operating system itself be (significantly) modified and is thus not a trivial undertaking.

- Thread-create and friends are now system calls and hence much slower than with user-mode threads. They are, however, still much faster than creating/switching/etc processes since there is so much shared state that does not need to be recreated.
- A thread that blocks causes no particular problem. The kernel can run another thread from this process or can run another process.
- Similarly a page fault, or infinite loop in one thread does not automatically block the other threads in the process.

Hybrid Implementations:

One can write a (user-level) thread library even if the kernel also has threads. This is sometimes called the M:N model since M user mode threads run on each of N kernel threads. Then each kernel thread can switch between user level threads. Thus switching between user-level threads within one kernel thread is very fast (no context switch) and we maintain the advantage that a blocking system call or page fault does not block the entire multi-threaded application since threads in other processes of this application are still runnable.

IPC:

- Many operating systems provide mechanisms for interprocess communication (IPC)
- Processes must communicate with one another in multiprogrammed and networked environments
- For example, a Web browser retrieving data from a distant server
- Essential for processes that must coordinate activities to achieve a common goal

Message-based interprocess communication

- Messages can be passed in one direction at a time
 - One process is the sender and the other is the receiver
- Message passing can be bidirectional
 - Each process can act as either a sender or a receiver
- Messages can be blocking or nonblocking

- Blocking requires the receiver to notify the sender when the message is received
- Nonblocking enables the sender to continue with other processing
- Popular implementation is a pipe
- A region of memory protected by the OS that serves as a buffer, allowing two or more processes to exchange data

15. Explain in detail about Security?

The security of NTFS volume is derived from the windows XP object model. Each NTFS file references a security description which contains

- Access token of the owner of the file.
- Access control list.
- Access privileges.

Traversal checks are inherently more expensive.

1. Volume management and fault tolerance:

- Ftdisk is the fault tolerant disk driver.
- Ftdisk provides several ways to combine multiple disk drives into one logical volume.
- It improves performances, capacity or reliability.

2. Volume set:

- In windows XP logical volume is called a volume set.
- It is one of the way to combine multiple disk.
- They concatenate all multiple disks logically to form large logical volume.
- It consist of 32 physical partitions.
- It can be extended without disturbing the data already stored in file system.

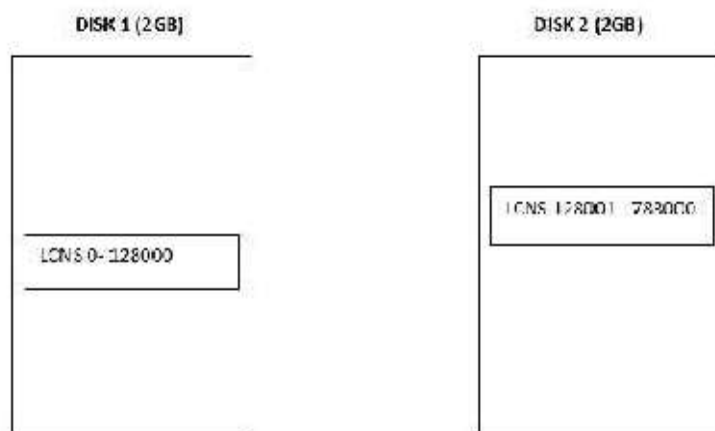


Fig 5.6 Volume set of two drives

3.Stripe set:

- It is another way to combine multiple physical partitions and to interleave their blocks in round robin fashion.
- It is also called as disk stription or RAID level 0.
- Ft disk uses a stripe set of 64 KBS.
- A stripe set forms one large logical volume but the physical layout can improve the I/O bandwidth, for a large I/O all the disk can transfer data in parallel.

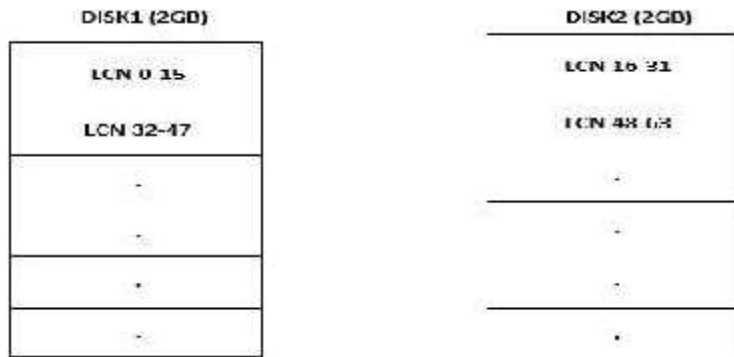


Fig 5.7 Stripe set on two drives

4.Stripe set with parity:

- Stripe set with parity is also called as RAID level 5.
- If the stripe set has eight disks then, for each of the seven data strips, on seven separate disks, there is a parity stripe in the eight disks.
- The parity stripe contains the bit-wise exclusive OR of the data stripes.
- If any one of the data stripe is destroyed the system can reconstruct the data by calculating the exclusive OR of the remaining seven.
- The data loss occurs very less.
- As all the seven data stripes have seven parity the I/O load is high.

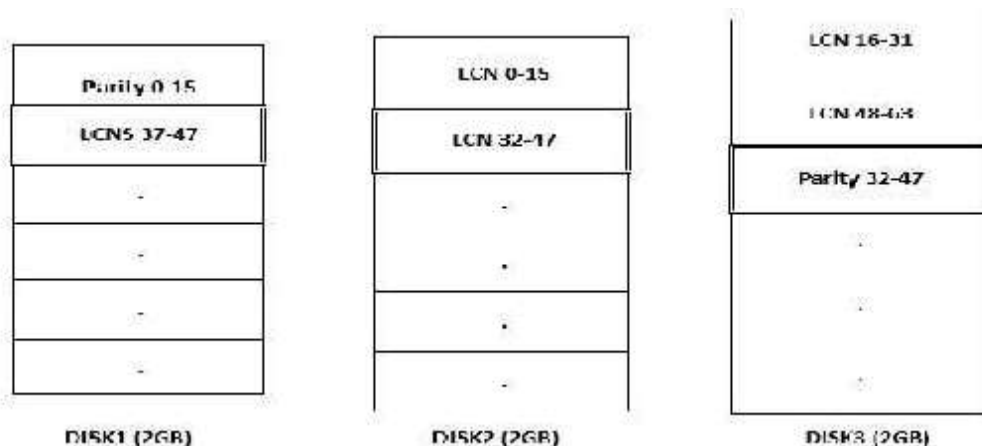


Fig 5.8 Stripe set with parity

5. Data Mirroring:

- Disk mirroring is also called as mirror sets or RAID level1 or duplex sets.
- A mirror set comprises of two equal sized partitions on two disks such that their data are identical.
- When an application writes data to a mirror set, Ftdisk writes the data in both partition.
- If one partition fails Ftdisks has another copy safely stored on the mirror.
- It improves the performance.
- We can attach the disks of a mirror set to separate disk controller. This arrangement is called duplex set.

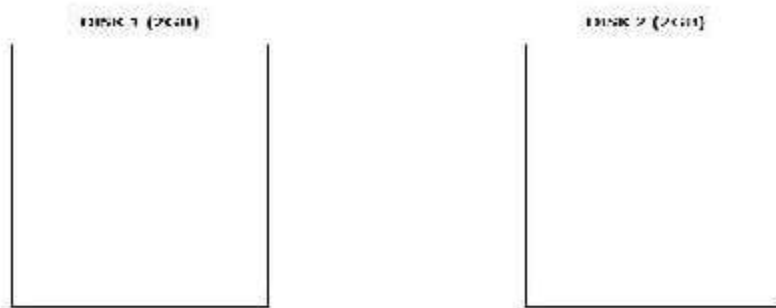


Fig 5.9 Mirror set on two drives

6. Sector sparing and cluster remapping:

- Disk uses a hardware technique called sector sparing.
- When a disk is formatted, it create a map from logical block number to good sector on the disk.
- If a sector fails, disk instructs the disk drive to substitute a space.
- NTFS uses a software technique called cluster remapping.
- If a disk block goes bad, NTFS substitutes a different unallocated block by changing any affected pointer in the MFT.
- If read fails the missing data is reconstructed and stored into a new location that is obtained by sector sparing or cluster remapping.

7. Compression and Encryption:

- NTFS can perform data compression on individual files or on all data files in a directory.
- NTFS divides the files data into compression units, which are blocks of 16 contiguous clusters.
- When each compression unit is written, a data compression algorithm is used.

- NTFS supports encryption of files.
- In this technique it provides security to the data using encryption algorithm.
- At the next end the message can be retrieved using decryption algorithm.

16. Write in detail about Kernel I/O subsystem

Kernels provide many services related to I/O. Several services-scheduling,buffering, caching, spooling, device reservation, and error handling-are provided by the kernel's I/O subsystem and build on the hardware and devicedriverinfrastructure. The I/O subsystem is also responsible for protecting itselffrom errant processes and malicious users.

I/O Scheduling

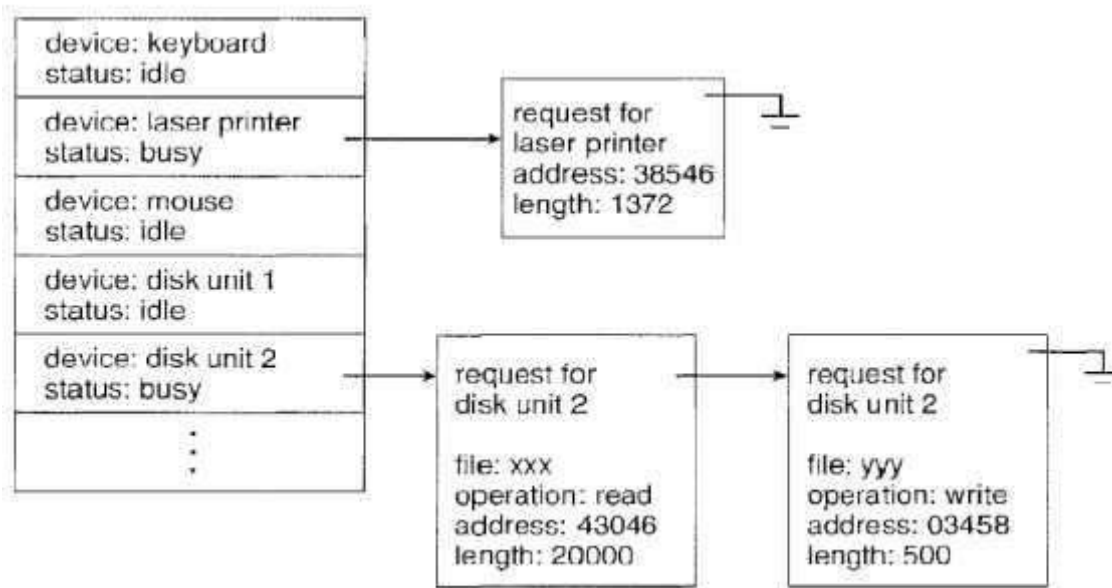
To schedule a set of I/O requests means to determine a good order in which toexecute them. The order in which applications issue system calls rarely is thebest choice. Scheduling can improve overall system performance, can sharedevice access fairly among processes, and can reduce the average waiting timefor I/O to complete. Here is a simple example to illustrate. Suppose that a disk arm is near the begilming of a disk and that three applications issue blockingread calls to that disk Application 1 requests a block near the end of the disk,application 2 requests one near the beginning, and application 3 requests onein the middle of the disk The operating system can reduce the distance that the disk arm travels by serving the applications in the order 2, 3, 1. Rearranging the order of service in this way is the essence of I/O scheduling.

Operating-system developers implement scheduling by maintaining a waitqueue of requests for each device. When an application issues a blocking I/Osystem call, the request is placed on the queue for that device. The I/O schedulerrearranges the order of the queue to improve the overall system efficiencyand the average response time experienced by applications. The operating system may also try to be fair, so that no one application receives especially poor service, or it may give priority service for delay-sensitive requests. For instance, requests from the virtual memory subsystem may take priority over application requests

When a kernel supports asynchronous I/O, it must be able to keep trackof many I/O requests at the same time. For this purpose, the operating systemmight attach the wait queue to a device status table. The kernel manages thistable, which contains an entry for each I/O device, as shown below.

Each table entry indicates the device's type, address, and state (not functioning,idle, or busy). If the device is busy with a request, the type of request and otherparameters will be stored in the table entry for that device.One way in which the I/O subsystem improves the efficiency of

the computer is by scheduling I/O operations. Another way is by using storage space in main memory or on disk via techniques called buffering, caching, and spooling.



Buffering

A *buffer* is a memory area that stores data being transferred between two devices or between a device and an application. Buffering is done for three reasons. One reason is to cope with a speed mismatch between the producer and consumer of a data stream. Suppose, for example, that a file is being received via modem for storage on the hard disk. The modem is about a thousand times slower than the hard disk. So a buffer is created in main memory to accumulate the bytes received from the modem. When an entire buffer of data has arrived, the buffer can be written to disk in a single operation. Since the disk write is not instantaneous and the modem still needs a place to store additional incoming data, two buffers are used. After the modem fills the first buffer, the disk write is requested. The modem then starts to fill the second buffer while the first buffer is written to disk. By the time the modem has filled the second buffer, the disk write from the first one should have completed, so the modem can switch back to the first buffer while the disk writes the second one. This double buffering decouples the producer of data from the consumer, thus relaxing timing requirements between them.

A second use of buffering is to provide adaptations for devices that have different data-transfer sizes. Such disparities are especially common in computer networking, where buffers are used widely for fragmentation and reassembly of messages. At the sending side, a large message is fragmented into small network packets. The packets are sent over the network, and the receiving side places them in a reassembly buffer to form an image of the source data.

A third use of buffering is to support copy semantics for application I/O. An example will clarify the meaning of "copy semantics." Suppose that an application has a buffer of data that it wishes to write to disk. It calls the `write()` system call providing a pointer to the buffer and an integer specifying the number of bytes to write. After the system call returns, what happens if the application changes the contents of the buffer? With copy semantics the version of the data written to disk is guaranteed to be the version at the time of the application system call independent of any subsequent changes in the application's buffer. A simple way in which the operating system can guarantee copy semantics is for the `write()` system call to copy the application data into a kernel buffer before returning control to the application. The disk write is performed from the kernel buffer, so that subsequent changes to the application buffer have no effect. Copying of data between kernel buffers and application data space is common in operating systems, despite the overhead that this operation introduces, because of the clean semantics. The same effect can be obtained more efficiently by clever use of virtual memory mapping and copy-on-write page protection.

Caching

A cache is a region of fast memory that holds copies of data. Access to the cached copy is more efficient than access to the original. For instance, the instructions of the currently running process are stored on disk, copied into physical memory, and copied again into the CPU's secondary and primary caches. The difference between a buffer and a cache is that a buffer may hold the only existing copy of a data item, whereas a cache, by definition, holds a copy on faster storage of an item that resides elsewhere.

Spooling and Device Reservation

A spool is a buffer that holds output for a device, such as a printer, that cannot accept interleaved data streams. Although a printer can serve only one job at a time, several applications may wish to print their output concurrently, without having their output mixed together. The operating system solves this problem by intercepting all output to the printer. Each application's output is spooled to a separate disk file. When an application finishes printing, the spooling system queues the corresponding spool file for output to the printer. The spooling system copies the queued spool files to the printer one at a time. In some operating systems, spooling is managed by a system daemon process. In others, it is handled by an in-kernel thread. In either case, the operating system provides a control interface that enables users and system administrators to display the queue, remove unwanted jobs before those jobs print, suspend printing while the printer is serviced, and so on.

Error handling

An operating system that uses protected memory can guard against many kinds of hardware and application errors, so that a complete system failure is not the usual result of each minor mechanical glitch. Devices and I/O transfers can fail in many ways, either for transient reasons, as when a network becomes overloaded, or for "permanent" reasons, as when a disk controller becomes defective. Operating systems can often compensate effectively for transient failures. For instance, a disk read() failure results in a read() retry, and a network send() error results in a resend(), if the protocol so specifies. Unfortunately, if an important component experiences a permanent failure, the operating system is unlikely to recover.

17. Discuss on file system structure in the linux systems. (NOV '15)

File Systems

Linux retains UNIX's standard file-system model. In UNIX, a file does not have to be an object stored on disk or fetched over a network from a remote file server. Rather, UNIX files can be anything capable of handling the input or output of a stream of data. Device drivers can appear as files, and interprocess communication channels or network connections also look like files to the user. The Linux kernel handles all these types of file by hiding the implementation details of any single file type behind a layer of software, the virtual file system (VFS).

The Virtual File System

The Linux VFS is designed around object-oriented principles. It has two components: a set of definitions that specify what file-system objects are allowed to look like and a layer of software to manipulate the objects. The VFS defines four main object types:

- An **inode object** represents an individual file.
- A **file object** represents an open file.
- A **superblock object** represents an entire file system.
- A **dentry object** represents an individual directory entry.

For each of these four object types, the VFS defines a set of operations. Every object of one of these types contains a pointer to a function table. The function table lists the addresses of the actual functions that implement the defined operations for that object.

- `int open (..)` — Open a file.
- `ssize_t read(..)` — Read from a file.
- `ssize_t write (..)` — Write to a file.
- `int mmap (..)` — Memory-map a file.

The complete definition of the file object is specified in the `struct file_operations`, which is located in the file `/usr/include/linux/fs.h`. An implementation of the file object (for a specific file type) is required to implement each function specified in the definition of the file object. The VFS software layer can perform an operation on one of the file-system objects by calling the appropriate function from the object's function table, without having to know in advance exactly what kind of object it is dealing with. The VFS does not know, or care, whether an inode represents a networked file, a disk file, a network socket, or a directory file. The appropriate function for that file's read operation will always be at the same place in its function table, and the VFS software layer will call that function without caring how the data are actually read.

The inode and file objects are the mechanisms used to access files. An inode object is a data structure containing pointers to the disk blocks that contain the actual file contents, and a file object represents a point of access to the data in an open file. A process cannot access an inode's contents without first obtaining a file object pointing to the inode. The file object keeps track of where in the file the process is currently reading or writing, to keep track of sequential file I/O.

The Linux ext2fs File System

The standard on-disk file system used by Linux is called **ext2fs**, for historical reasons. Linux was originally programmed with a Minix-compatible file system, to ease exchanging data with the Minix development system, but that file system was severely restricted by 14-character file-name limits and a maximum file-system size of 64 MB. The Minix file system was superseded by a new file system, which was christened the **extended file system (extfs)**. A later redesign of this file system to improve performance and scalability and to add a few missing features led to the **second extended file system (ext2fs)**.

It uses a similar mechanism for locating the data blocks belonging to a specific file, storing data-block pointers in indirect blocks throughout the file system with up to three levels of indirection. As in FFS, directory files are stored on disk just like normal files, although their contents are interpreted differently.

Each block in a directory file consists of a linked list of entries; each entry contains the length of the entry, the name of a file, and the inode number of the inode to which that entry refers. The main differences between ext2fs and FFS lie in their disk-allocation policies. In FFS, the disk is allocated to files in blocks of 8 KB. These blocks are subdivided into fragments of 1 KB for storage of small files or partially filled blocks at the ends of files. In contrast, ext2fs does not use fragments at all but performs all its allocations in smaller units. The default block size on ext2fs is 1 KB, although 2-KB and 4-KB blocks are also supported.

To maintain high performance, the operating system must try to perform I/O operations in large chunks whenever possible by clustering physically adjacent I/O requests. Clustering reduces the per-request overhead incurred by device drivers, disks, and disk-controller hardware.

Journaling

Many different types of file systems are available for Linux systems. One popular feature in a file system is **journaling**, whereby modifications to the filesystem are sequentially written to a journal. A set of operations that performs a specific task is a **transaction**. Once a transaction is written to the journal, it is considered to be committed, and the system call modifying the file system (i.e. write ()) can return to the user process, allowing it to continue execution. Meanwhile, the journal entries relating to the transaction are replayed across the actual file-system structures. As the changes are made, a pointer is updated to indicate which actions have completed and which are still incomplete.

When an entire committed transaction is completed, it is removed from the journal. The journal, which is actually a circular buffer, may be in a separate section of the file system, or it may even be on a separate disk spindle. It is more efficient, but more complex, to have it under separate read-write heads, thereby decreasing head contention and seek times.

If the system crashes, there will be zero or more transactions in the journal. Those transactions were never completed to the file system even though they were committed by the operating system, so they must be completed. The transactions can be executed from the pointer until the work is complete, and the file-system structures remain consistent. The only problem occurs when a transaction has been aborted. That is, it was not committed before the system crashed. Any changes from those transactions that were applied to the filesystem must be undone, again preserving the consistency of the file system. This recovery is all that is needed after a crash, eliminating all problems with consistency checking.

The Linux proc File System

The flexibility of the Linux VFS enables us to implement a file system that does not store data persistently at all but rather simply provides an interface to some other functionality. The Linux **process file system**, known as the /proc file system, is an example of a file system whose contents are not actually stored anywhere but are computed on demand according to user file I/O requests. A /proc file system is not unique to Linux. SVR4 UNIX introduced a /proc file system as an efficient interface to the kernel's process debugging support:

Each subdirectory of the file system corresponded not to a directory on any disk but rather to an active process on the current system. A listing of the filesystem reveals one directory per process,

with the directory name being the ASCII decimal representation of the process's unique process identifier (PID).

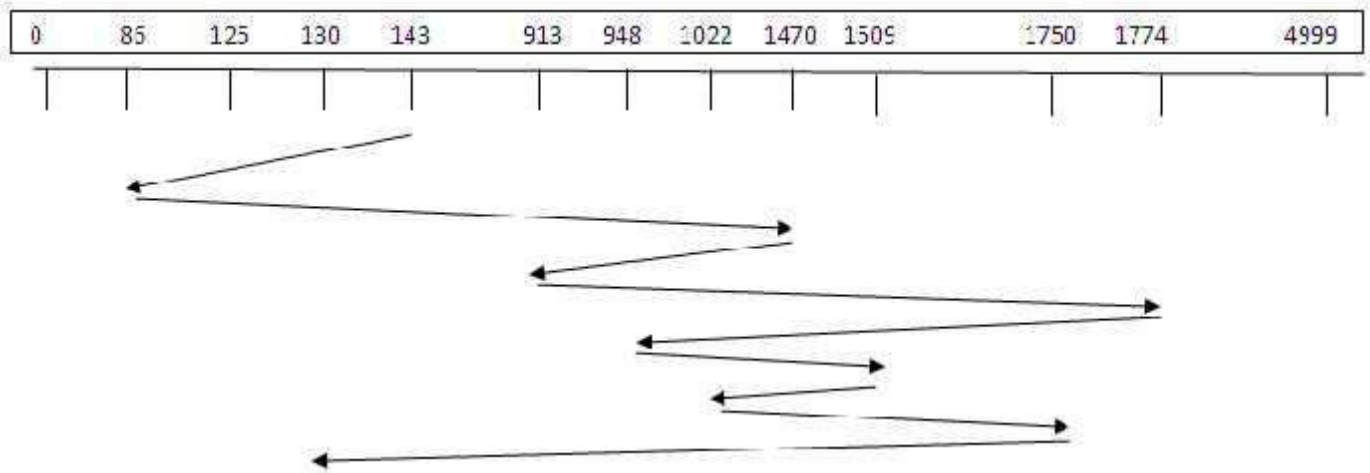
Linux implements such a /proc file system but extends it greatly by adding a number of extra directories and text files under the file system's root directory. These new entries correspond to various statistics about the kernel and the associated loaded drivers. The /proc file system provides a way for programs to access this information as plain text files, which the standard UNIX user environment provides powerful tools to process. For example, in the past, the traditional UNIX `ps` command for listing the states of all running processes has been implemented as a privileged process that reads the process state directly from the kernel's virtual memory. Under Linux, this command is implemented as an entirely unprivileged program that simply parses and formats the information from /proc. The /proc file system must implement two things: a directory structure and the file contents within.

18. Suppose that a disk drive has 5000 cylinders numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests in FIFO order is: 86, 1470, 913, 1774, 948, 1509, 1022, 1750, 130

Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms? (Nov 16) (Apr 17)

- (a) FCFS**
- (b) SSTF**
- (c) SCAN**
- (d) LOOK**

FCFS

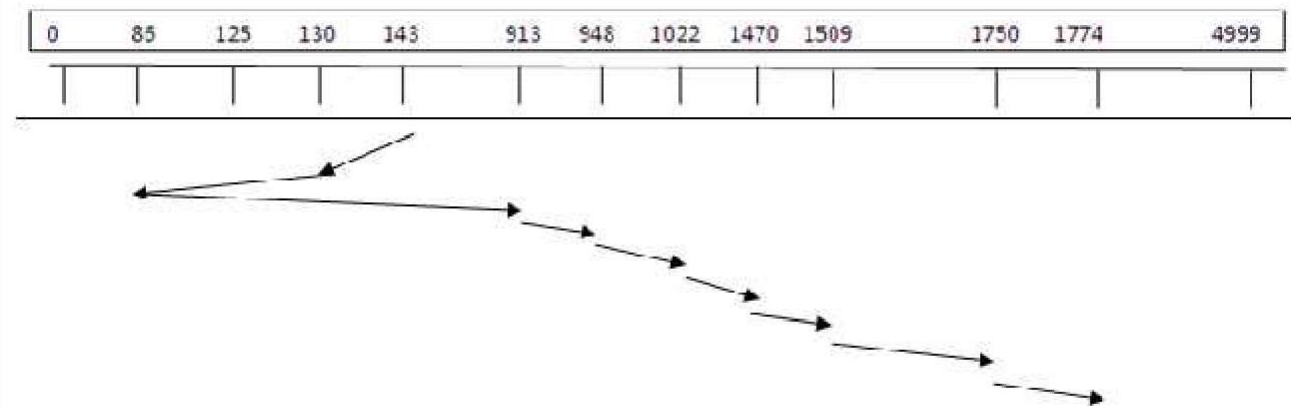


Distances

143 to 86=57; 86 to 1470=1384; 1470 to 913 =557; 913 to 1774=861; 1774 to 948=826;
948 to 1509=561; 1509 to 1022=487; 1022 to 1750=728; 1750 to 130=1620;

Total distance moved by the disk arm=7081

SSTF

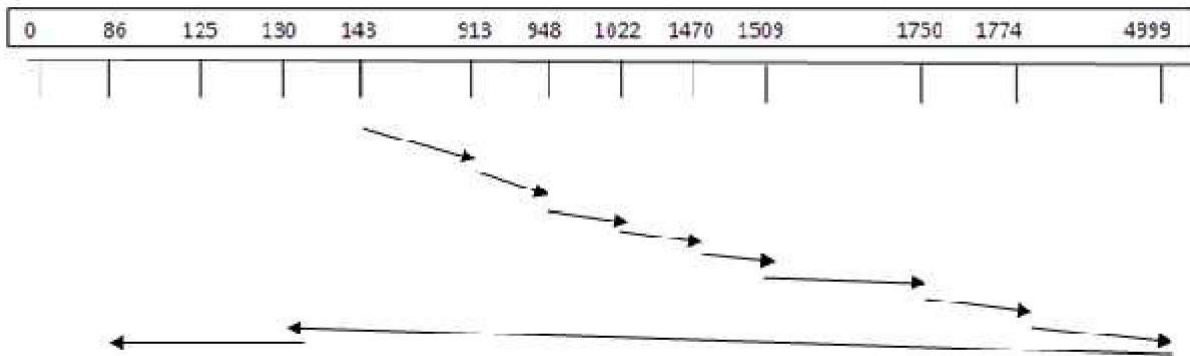


Distances

143 to 130=13; 130 to 86=44; 86 to 913=827; 913 to 948= 35; 948 to 1022=74; 1022 to
1470=448; 1470 to 1509=39; 1509 to 1750=241; 1750 to 1774=24;

Total distance moved by the disk arm=1745

SCAN

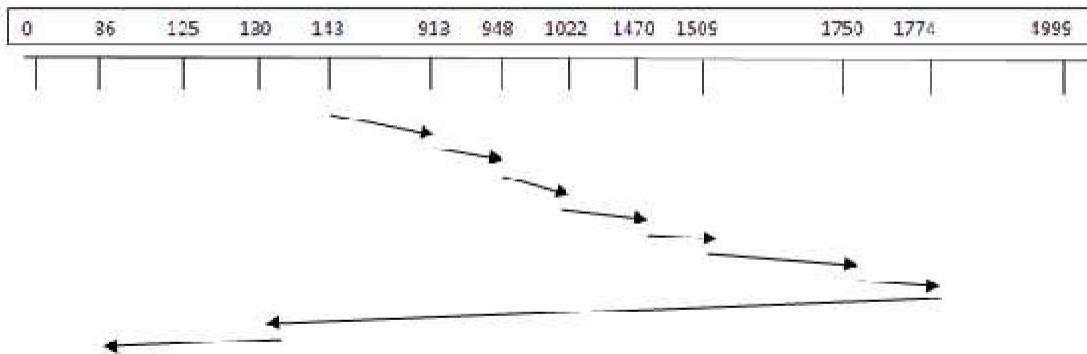


Distances

143 to 913=770; 913 to 948= 35; 948 to 1022=74; 1022 to 1470=448; 1470 to 1509=39; 1509 to 1750=241; 1750 to 1774=24; 1774 to 4999= 3225; 4999 to 130 = 4869; 130 to 86 = 44

Total distance moved by the disk arm=9769

LOOK



Distances

143 to 913=770; 913 to 948= 35; 948 to 1022=74; 1022 to 1470=448; 1470 to 1509=39; 1509 to 1750=241; 1750 to 1774=24; 1774 to 130 = 1644; 130 to 86 = 44

Total distance moved by the disk arm= 3319

Pondicherry University Questions

2 MARKS

1. Define Bootstrap Program?(NOV'12) (Ref.Pg.No.1 Qn.No.1)
2. What do you mean by seek time? (APR'12) (Nov 16) (Apr 17)(Ref.Pg.No.4 Qn.No.18)
3. List the disk performance parameter?(APR'13) (Ref.Pg.No.4 Qn.No.19)
4. What is the purpose of device drivers? (APR'13) (Ref.Pg.No.4 Qn.No.20)
5. List the role of process manager?(NOV'13) (Ref.Pg.No.12 Qn.No.52)
6. What is IPC? (APR'12) (Ref.Pg.No.9 Qn.No.40)
7. Write any two advantages of Linux?(NOV'12) (Ref.Pg.No.10 Qn.No.43)
8. Brief about Process Environment Block (PEB).(NOV'13) (Ref.Pg.No.11 Qn.No.49)
9. Sketch the components of a Linux system (NOV'14) (Ref.Pg.No.16 Qn.No.68)
10. Define Kernel in OS? (APR'12) (APR'15) (NOV '15) (Ref.Pg.No.7 Qn.No.32)
11. What type of security is provided by Windows XP (APR'15) (Ref.Pg.No.14 Qn.No.61)
12. Define physical record and logical record. (NOV '15)(Ref.Pg.No.16 Qn.No.69)
13. Define latency time (Apr 17) (Ref.Pg.No.16 Qn.No.70)
14. What are the advantages of linked allocation? (Apr 17) (Ref.Pg.No.16 Qn.No.71)
15. What is swapping? What is the significant of swap space management? (Nov 16) (Ref.Pg.No.17 Qn.No.72)
16. What is the use of boot block? (Apr 16) (Ref.Pg.No.17 Qn.No.73)
17. What are the functions of Virtual File System (VFS)? (Apr 16) (Ref.Pg.No.17 Qn.No.74)

11 Marks:

1. Explain Disk Scheduling in detail with example? (APR '11) (APR '13) (NOV '15)(Ref.Pg.No.19 Qn.No.2)
2. Discuss the various techniques used to improve the efficiency and performance of secondary storage? 6 Marks (APR '14) (Ref.Pg.No.22 Qn.No.3)
3. Briefly explain Disk management and swap space management? (NOV '12)(Apr 16)(Ref.Pg.No.23 Qn.No.4)
4. Explain in detail about Kernel architecture of Linux? (APR'13)(NOV'13) (Ref.Pg.No.24 Qn.No.5)
5. Explain in Detail about Process management? (NOV'14) (Ref.Pg.No 27 Qn.No.6)
6. Explain how interprocess communication is carried out in Linux OS? (NOV'12) (APR'13) (APR'15) (Ref.Pg.No.30 Qn.No.8)

7. Explain how the memory and file management is implemented in windows XP? (APR'13)
(Ref.Pg.No.33 Qn.No.10)
8. Describe about memory allocation stages. (11)(NOV'13) (Ref.Pg.No.40 Qn.No.13)
- 9 Explain how the following mechanisms are implemented in Windows XP?(1)Thread (2) IPC
(APR'12) (Ref.Pg.No.43 Qn.No.14)
10. Explain in detail about Security? (NOV'14) (Ref.Pg.No.34 Qn.No.11)
11. Explain the allocation methods for disk space in file allocation? (11) (NOV 12, 13) (APR
'11)(APR'15) (Apr 16)(Nov 16)(Apr 17) (Ref.Pg.No.17 Qn.No.1)
12. Explain in detail about Windows XP System Architecture?(NOV'12) (APR'15) (Ref.Pg.No.34
Qn.No.12)
13. Discuss on file system structure in the linux systems. (NOV '15) (Ref.Pg.No.51 Qn.No.17)
14. Suppose that a disk drive has 5000 cylinders numbered 0 to 4999. The drive is currently serving a request at cylinder 143, and the previous request was at cylinder 125. The queue of pending requests in FIFO order is: 86, 1470, 913, 1774,948,1509,1022,1750,130Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms? (Nov 16) (Apr 17)
(Ref.Pg.No.56 Qn.No.18)
- (a) FCFS
 - (b) SSTF
 - (c) SCAN
 - (d) LOOK